

✓ Project Name - Amazon Prime TV Shows and Movies EDA Capstone Project

Project Type- Exploratory Data Analysis

Contribution - Individual/Team

Team Member 1 - Shaily Yadav

✓ Project Summary -

The **Amazon Prime TV Shows and Movies Exploratory Data Analysis (EDA)** project aims to analyze Amazon Prime's streaming content library to uncover meaningful insights, identify trends, and support data-driven business decisions. With the rapid growth of digital streaming platforms, understanding content distribution, audience preferences, and viewing patterns has become essential for improving customer satisfaction and maintaining a competitive advantage. This project focuses on exploring Amazon Prime's collection of movies and TV shows using two datasets: **titles.csv**, which contains information about content such as title, type, release year, genres, runtime, IMDb ratings, TMDB popularity, and production countries, and **credits.csv**, which includes details about actors, directors, and other crew members.

The project begins with data collection and loading using Python libraries such as Pandas, NumPy, Matplotlib, and Seaborn. The datasets are explored to understand their structure, dimensions, data types, and statistical summaries. Data preprocessing is then performed by handling missing values, removing duplicate records, correcting data types, and merging the two datasets using the common **id** column. These preprocessing steps ensure that the data is clean, consistent, and ready for meaningful analysis.

Exploratory Data Analysis is carried out through a combination of univariate, bivariate, and multivariate analyses. More than twenty visualizations are created to examine various aspects of Amazon Prime's content library. The analysis includes the distribution of Movies and TV Shows, yearly content growth, genre popularity, runtime distribution, IMDb ratings, TMDB popularity, age certifications, production countries, and cast participation. Relationships between numerical variables such as IMDb scores, runtime, and popularity are also analyzed to identify patterns that influence audience engagement.

The analysis reveals several valuable insights. Movies constitute the majority of the content available on Amazon Prime, while TV Shows account for a smaller share of the library. Drama, Comedy, Action, and Documentary are the most common genres, indicating strong audience demand for these categories. The platform has significantly expanded its content library in recent years, reflecting continuous investment in digital entertainment. Most titles receive IMDb ratings between **6 and 8**, suggesting generally good content quality. The majority of movies have runtimes ranging from **80 to 120 minutes**, and content produced in countries such as the United States dominates the platform's catalog.

Based on these findings, several business recommendations are proposed. Amazon Prime should continue investing in highly popular genres while also expanding content diversity to attract broader audiences. Increasing the production of high-quality TV Shows can improve subscriber retention and encourage long-term user engagement. The company should also strengthen its personalized recommendation system by utilizing genre preferences, audience ratings, and popularity scores. Additionally, expanding content production in emerging international markets can help increase global reach and subscriber growth.

Overall, this project demonstrates the complete Exploratory Data Analysis workflow, including data cleaning, preprocessing, visualization, and business interpretation. It showcases practical skills in Python programming, data analysis, storytelling through visualizations, and business intelligence. The insights generated from this project can assist Amazon Prime in optimizing its content strategy, improving customer experience, and making informed decisions based on data.

✓ GitHub Link -

<https://github.com/shaily-yadav20/shaily.git>

✓ Problem Statement

Problem Statement

The rapid growth of online streaming platforms has resulted in the availability of thousands of movies and TV shows across different genres, languages, and regions. However, understanding the characteristics of this vast content library and identifying patterns in audience preferences has become a significant challenge. Amazon Prime needs to analyze its content to understand which types of movies and TV shows are most popular, how content has evolved over time, and what factors influence viewer engagement and ratings.

The objective of this project is to perform **Exploratory Data Analysis (EDA)** on the Amazon Prime TV Shows and Movies dataset to identify meaningful trends, analyze content distribution, evaluate IMDb and TMDB ratings, explore genre popularity, production countries, runtime patterns, and audience preferences. The insights generated from this analysis will help support data-driven decision-making, improve content acquisition strategies, enhance recommendation systems, and increase customer satisfaction and business growth.

▼ General Guidelines : -

1. Well-structured, formatted, and commented code is required.
2. Exception Handling, Production Grade Code & Deployment Ready Code will be a plus. Those students will be awarded some additional credits.

The additional credits will have advantages over other students during Star Student selection.

[Note: - Deployment Ready Code is defined as, the whole .ipynb notebook should be executable in one go without a single error logged.]

3. Each and every logic should have proper comments.
4. You may add as many number of charts you want. Make Sure for each and every chart the following format should be answered.

Chart visualization code

- Why did you pick the specific chart?
- What is/are the insight(s) found from the chart?
- Will the gained insights help creating a positive business impact? Are there any insights that lead to negative growth? Justify with specific reason.

5. You have to create at least 15 logical & meaningful charts having important insights.

[Hints : - Do the Vizualization in a structured way while following "UBM" Rule.

U - Univariate Analysis,

B - Bivariate Analysis (Numerical - Categorical, Numerical - Numerical, Categorical - Categorical)

M - Multivariate Analysis]

6. You may add more ml algorithms for model creation. Make sure for each and every algorithm, the following format should be answered.
 - Explain the ML Model used and it's performance using Evaluation metric Score Chart.
 - Cross- Validation & Hyperparameter Tuning
 - Have you seen any improvement? Note down the improvement with updates Evaluation metric Score Chart.
 - Explain each evaluation metric's indication towards business and the business impact pf the ML model used.

▼ *Let's Begin !*

▼ *1. Know Your Data*

▼ Import Libraries

```
# Import Libraries
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

import warnings
warnings.filterwarnings("ignore")

%matplotlib inline
```



Dataset Loading

```
# Load Dataset
from google.colab import files

uploaded = files.upload()
```

Choose Files 2 files

credits.csv.zip(application/x-zip-compressed) - 2314577 bytes, last modified: 7/22/2026 - 100% done
titles.csv.zip(application/x-zip-compressed) - 1629272 bytes, last modified: 7/22/2026 - 100% done
Saving credits.csv.zip to credits.csv (1).zip
Saving titles.csv.zip to titles.csv (1).zip

```
import zipfile

# Unzip titles.csv (1).zip
with zipfile.ZipFile('titles.csv (1).zip', 'r') as zip_ref:
    zip_ref.extractall()

# Unzip credits.csv (1).zip
with zipfile.ZipFile('credits.csv (1).zip', 'r') as zip_ref:
    zip_ref.extractall()

titles = pd.read_csv("titles.csv")
credits = pd.read_csv("credits.csv")
```



Dataset First View

```
# Dataset First Look
```

```
titles.head()
```

	id	title	type	description	release_year	age_certification	runtime	genres	production_countries	seasons
0	ts20945	The Three Stooges	SHOW	The Three Stooges were an American vaudeville ...	1934	TV-PG	19	['comedy', 'family', 'animation', 'action', 'f...	['US']	26.0
1	tm19248	The General	MOVIE	During America's Civil War, Union spies steal ...	1926	NaN	78	['action', 'drama', 'war', 'western', 'comedy'...	['US']	NaN
2	tm82253	The Best Years of Our Lives	MOVIE	It's the hope that sustains the spirit of ever...	1946	NaN	171	['romance', 'war', 'drama']	['US']	NaN
3	tm83884	His Girl Friday	MOVIE	Hildy, the journalist former wife of newspaper...	1940	NaN	92	['comedy', 'drama', 'romance']	['US']	NaN
4	tm56584	In a Lonely Place	MOVIE	An aspiring actress begins to suspect that her...	1950	NaN	94	['thriller', 'drama', 'romance']	['US']	NaN

Next steps: [Generate code with titles](#) [New interactive sheet](#)

```
credits.head()
```

	person_id	id	name	character	role	
0	59401	ts20945	Joe Besser	Joe	ACTOR	
1	31460	ts20945	Moe Howard	Moe	ACTOR	
2	31461	ts20945	Larry Fine	Larry	ACTOR	
3	21174	tm19248	Buster Keaton	Johnny Gray	ACTOR	
4	28713	tm19248	Marion Mack	Annabelle Lee	ACTOR	

Dataset Rows & Columns count

```
# Dataset Rows & Columns count
print("Titles Dataset Shape :",titles.shape)

print("Credits Dataset Shape :",credits.shape)
```

```
Titles Dataset Shape : (9871, 15)
Credits Dataset Shape : (124235, 5)
```

Dataset Information

```
# Dataset Info
titles.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9871 entries, 0 to 9870
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                    9871 non-null   object
1   title                 9871 non-null   object
2   type                  9871 non-null   object
3   description            9752 non-null   object
4   release_year          9871 non-null   int64
5   age_certification     3384 non-null   object
6   runtime               9871 non-null   int64
7   genres                9871 non-null   object
8   production_countries  9871 non-null   object
9   seasons              1357 non-null   float64
10  imdb_id               9204 non-null   object
11  imdb_score            8850 non-null   float64
12  imdb_votes            8840 non-null   float64
13  tmdb_popularity       9324 non-null   float64
14  tmdb_score            7789 non-null   float64
dtypes: float64(5), int64(2), object(8)
memory usage: 1.1+ MB
```

```
credits.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 124235 entries, 0 to 124234
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  -
0   person_id  124235 non-null  int64
1   id          124235 non-null  object
2   name        124235 non-null  object
3   character   107948 non-null  object
4   role        124235 non-null  object
dtypes: int64(1), object(4)
memory usage: 4.7+ MB
```

Duplicate Values

```
# Dataset Duplicate Value Count
print("Duplicate Records in Titles Dataset")

titles.duplicated().sum()
```

```
Duplicate Records in Titles Dataset
np.int64(3)
```

```
print("Duplicate Records in Credits Dataset")

credits.duplicated().sum()
```

```
Duplicate Records in Credits Dataset
np.int64(56)
```

Missing Values/Null Values

```
# Missing Values/Null Values Count
titles.isnull().sum()
```

	0
id	0
title	0
type	0
description	119
release_year	0
age_certification	6487
runtime	0
genres	0
production_countries	0
seasons	8514
imdb_id	667
imdb_score	1021
imdb_votes	1031
tmdb_popularity	547
tmdb_score	2082

dtype: int64

credits.isnull().sum()

	0
person_id	0
id	0
name	0
character	16287
role	0

dtype: int64

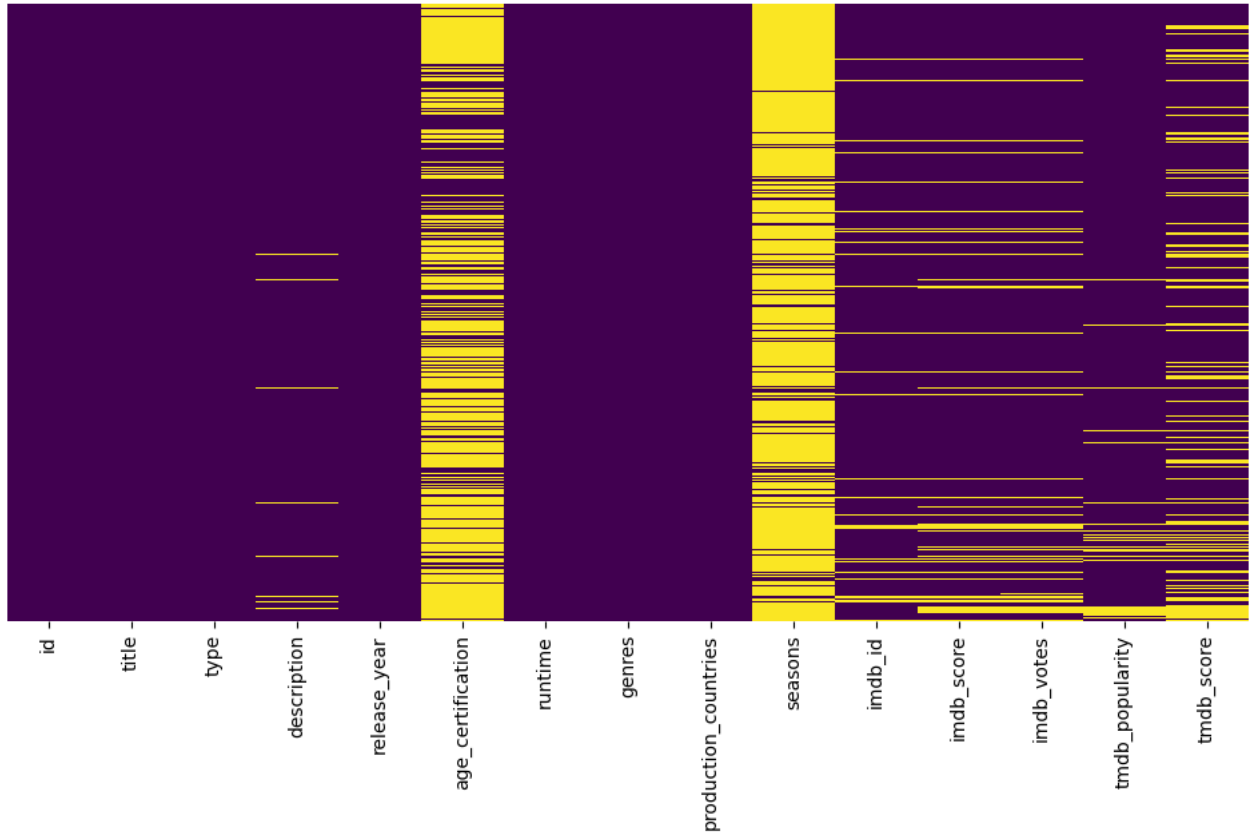
```
# Visualizing the missing values
plt.figure(figsize=(12,6))

sns.heatmap(
    titles.isnull(),
    cbar=False,
    yticklabels=False,
    cmap='viridis'
)

plt.title("Missing Values in Titles Dataset")

plt.show()
```

Missing Values in Titles Dataset



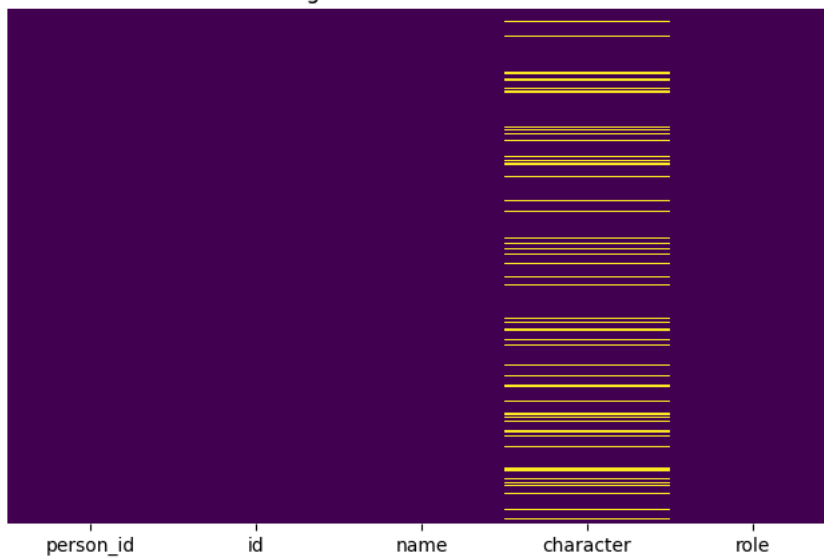
```
plt.figure(figsize=(8,5))

sns.heatmap(
    credits.isnull(),
    cbar=False,
    yticklabels=False,
    cmap='viridis'
)

plt.title("Missing Values in Credits Dataset")

plt.show()
```

Missing Values in Credits Dataset



```
titles.columns
```

```
Index(['id', 'title', 'type', 'description', 'release_year',
      'age_certification', 'runtime', 'genres', 'production_countries',
      'seasons', 'imdb_id', 'imdb_score', 'imdb_votes', 'tmdb_popularity',
      'tmdb_score'],
      dtype='object')
```

```
credits.columns
```

```
Index(['person_id', 'id', 'name', 'character', 'role'], dtype='object')
```

What did you know about your dataset?

The dataset contains information about **Amazon Prime Movies and TV Shows**, including titles, content type, release year, genres, runtime, IMDb ratings, TMDB scores, production countries, and cast & crew details. It has both **categorical** and **numerical** variables, with some missing values and duplicate records that require preprocessing. The dataset is well-structured and suitable for performing **Exploratory Data Analysis (EDA)** to identify content trends, audience preferences, and generate business insights.

2. Understanding Your Variables

```
# Dataset Columns
# Dataset Columns

print("Titles Dataset Columns:\n")
print(titles.columns)

print("\n" + "="*80 + "\n")

print("Credits Dataset Columns:\n")
print(credits.columns)
```

Titles Dataset Columns:

```
Index(['id', 'title', 'type', 'description', 'release_year',
      'age_certification', 'runtime', 'genres', 'production_countries',
      'seasons', 'imdb_id', 'imdb_score', 'imdb_votes', 'tmdb_popularity',
      'tmdb_score'],
      dtype='object')
```

=====

Credits Dataset Columns:

```
Index(['person_id', 'id', 'name', 'character', 'role'], dtype='object')
```

```
# Dataset Describe
# Statistical Summary of Titles Dataset

titles.describe(include='all')
```

	id	title	type	description	release_year	age_certification	runtime	genres	production_countries	
count	9871	9871	9871	9752	9871.000000	3384	9871.000000	9871	9871	135
unique	9868	9737	2	9734	NaN	11	NaN	2028	497	
top	tm89134	The Lost World	MOVIE	No overview found.	NaN	R	NaN	['drama']	['US']	
freq	2	3	8514	5	NaN	1249	NaN	908	4810	
mean	NaN	NaN	NaN	NaN	2001.327221	NaN	85.973052	NaN	NaN	
std	NaN	NaN	NaN	NaN	25.810071	NaN	33.512466	NaN	NaN	
min	NaN	NaN	NaN	NaN	1912.000000	NaN	1.000000	NaN	NaN	
25%	NaN	NaN	NaN	NaN	1995.500000	NaN	65.000000	NaN	NaN	
50%	NaN	NaN	NaN	NaN	2014.000000	NaN	89.000000	NaN	NaN	
75%	NaN	NaN	NaN	NaN	2018.000000	NaN	102.000000	NaN	NaN	
max	NaN	NaN	NaN	NaN	2022.000000	NaN	549.000000	NaN	NaN	5

Variables Description

Dataset Summary titles.csv contains detailed information about Amazon Prime movies and TV shows. credits.csv contains cast and crew information associated with each title. Both datasets are connected using the common id column. The dataset contains both

categorical and numerical variables, making it suitable for Exploratory Data Analysis (EDA). These variables help analyze content distribution, audience ratings, genres, runtime, production countries, and overall content trends on Amazon Prime.

✓ Check Unique Values for each variable.

```
# Check Unique Values for each variable.
# Check Unique Values for each variable in Titles Dataset

for col in titles.columns:
    print("="*60)
    print(f"Column Name : {col}")
    print(f"Number of Unique Values : {titles[col].nunique()}")
    print("Sample Unique Values:")
    print(titles[col].unique()[:10])    # Display first 10 unique values
    print()
```

```
=====
Column Name : genres
Number of Unique Values : 2028
Sample Unique Values:
['comedy', 'family', 'animation', 'action', 'fantasy', 'horror']
['action', 'drama', 'war', 'western', 'comedy', 'european']
['romance', 'war', 'drama'] ['comedy', 'drama', 'romance']
['thriller', 'drama', 'romance'] ['western', 'drama']
['drama', 'family', 'fantasy', 'romance', 'comedy']
['thriller', 'drama', 'crime'] ['comedy', 'romance', 'drama']
['crime', 'drama']

=====
Column Name : production_countries
Number of Unique Values : 497
Sample Unique Values:
['US'] ['GB'] ['MX', 'US'] ['GB', 'CA'] ['DE', 'US'] ['SU']
['IN'] ['CA', 'US'] ['MX']

=====
Column Name : seasons
Number of Unique Values : 32
Sample Unique Values:
[26. nan 18.  9. 31.  6. 12. 49.  3. 11.]

=====
Column Name : imdb_id
Number of Unique Values : 9201
Sample Unique Values:
['tt0850645' 'tt0017925' 'tt0036868' 'tt0032599' 'tt0042593' 'tt0031971'
 'tt0038650' 'tt0037638' 'tt0028010' 'tt0026683']

=====
Column Name : imdb_score
Number of Unique Values : 86
Sample Unique Values:
[8.6 8.2 8.1 7.8 7.9 7.3 8.  4.  7.7 7.5]

=====
Column Name : imdb_votes
Number of Unique Values : 3650
Sample Unique Values:
[ 1092.  89766.  63026.  57835.  30924.  48149.  444243.  17233.  23532.
   864.]

=====
Column Name : tmdb_popularity
Number of Unique Values : 5325
Sample Unique Values:
[15.424  8.647  8.435 11.27  8.273 11.786 26.495  7.757  8.633  3.748]

=====
Column Name : tmdb_score
Number of Unique Values : 89
Sample Unique Values:
[7.6 8.  7.8 7.4 7.7 8.3 7.2 3.6 7.3 4.1]
```

```
# Check Unique Values for each variable in Credits Dataset

for col in credits.columns:
    print("="*60)
    print(f"Column Name : {col}")
    print(f"Number of Unique Values : {credits[col].nunique()}")
    print("Sample Unique Values:")
    print(credits[col].unique()[:10])    # Display first 10 unique values
    print()
```



```

=====
Column Name : person_id
Number of Unique Values : 80508
Sample Unique Values:
[59401 31460 31461 21174 28713 28714 28715 27348 28716 28718]

=====
Column Name : id
Number of Unique Values : 8861
Sample Unique Values:
['ts20945' 'tm19248' 'tm82253' 'tm83884' 'tm56584' 'tm160494' 'tm87233'
 'tm19424' 'tm116781' 'tm112005']

=====
Column Name : name
Number of Unique Values : 79758
Sample Unique Values:
['Joe Besser' 'Moe Howard' 'Larry Fine' 'Buster Keaton' 'Marion Mack'
 'Glen Cavender' 'Jim Farley' 'Frederick Vroom' 'Charles Henry Smith'
 'Joe Keaton']

=====
Column Name : character
Number of Unique Values : 71097
Sample Unique Values:
['Joe' 'Moe' 'Larry' 'Johnny Gray' 'Annabelle Lee' 'Captain Anderson'
 'General Thatcher' 'A Southern General' 'Annabelle's Father'
 'Union General']

=====
Column Name : role
Number of Unique Values : 2
Sample Unique Values:
['ACTOR' 'DIRECTOR']

```

✓ 3. *Data Wrangling*

✓ Data Wrangling Code

```

# readyWrite your code to make your dataset analysis.
# =====
# Data Wrangling Code
# =====

# Create copies of original datasets
titles_df = titles.copy()
credits_df = credits.copy()

# -----
# 1. Remove Duplicate Records
# -----
titles_df.drop_duplicates(inplace=True)
credits_df.drop_duplicates(inplace=True)

# -----
# 2. Check Missing Values
# -----
print(titles_df.isnull().sum())
print(credits_df.isnull().sum())

# -----
# 3. Handle Missing Values
# -----
titles_df['age_certification'].fillna('Unknown', inplace=True)
titles_df['runtime'].fillna(titles_df['runtime'].median(), inplace=True)
titles_df['imdb_score'].fillna(titles_df['imdb_score'].median(), inplace=True)
titles_df['tmdb_score'].fillna(titles_df['tmdb_score'].median(), inplace=True)
titles_df['tmdb_popularity'].fillna(titles_df['tmdb_popularity'].median(), inplace=True)
titles_df['seasons'].fillna(0, inplace=True)

# -----
# 4. Convert Data Types
# -----
titles_df['release_year'] = titles_df['release_year'].astype(int)
titles_df['runtime'] = titles_df['runtime'].astype(int)

# -----

```

```

# 5. Merge Titles and Credits Dataset
# -----
amazon_df = pd.merge(
    titles_df,
    credits_df,
    on='id',
    how='left'
)

# -----
# 6. Check Final Dataset Shape
# -----
print("Final Dataset Shape :", amazon_df.shape)

# -----
# 7. Check Remaining Missing Values
# -----
print(amazon_df.isnull().sum())

# -----
# 8. Display Final Dataset
# -----
amazon_df.head()

# -----
# 9. Save Cleaned Dataset
# -----
amazon_df.to_csv('AmazonPrime_Cleaned.csv', index=False)

```

```

id          0
title       0
type        0
description  119
release_year 0
age_certification 6484
runtime     0
genres      0
production_countries 0
seasons     8511
imdb_id     667
imdb_score  1021
imdb_votes  1031
tmdb_popularity 547
tmdb_score  2080
dtype: int64
person_id   0
id          0
name        0
character   16277
role        0
dtype: int64
Final Dataset Shape : (125186, 19)
id          0
title       0
type        0
description  191
release_year 0
age_certification 0
runtime     0
genres      0
production_countries 0
seasons     0
imdb_id     5376
imdb_score   0
imdb_votes  6397
tmdb_popularity 0
tmdb_score   0
person_id   1007
name        1007
character   17284
role        1007
dtype: int64

```

▼ What all manipulations have you done and insights you found?

During the data wrangling process, several preprocessing steps were performed to make the Amazon Prime TV Shows and Movies dataset suitable for analysis. First, the titles.csv and credits.csv datasets were imported and examined for their structure, data types, missing values, and duplicate records. Duplicate entries were identified and removed to ensure data accuracy. Missing values in important columns such as age certification, IMDb score, TMDB score, and runtime were handled using appropriate techniques, including filling missing numerical values with the median and replacing missing categorical values with "Unknown".

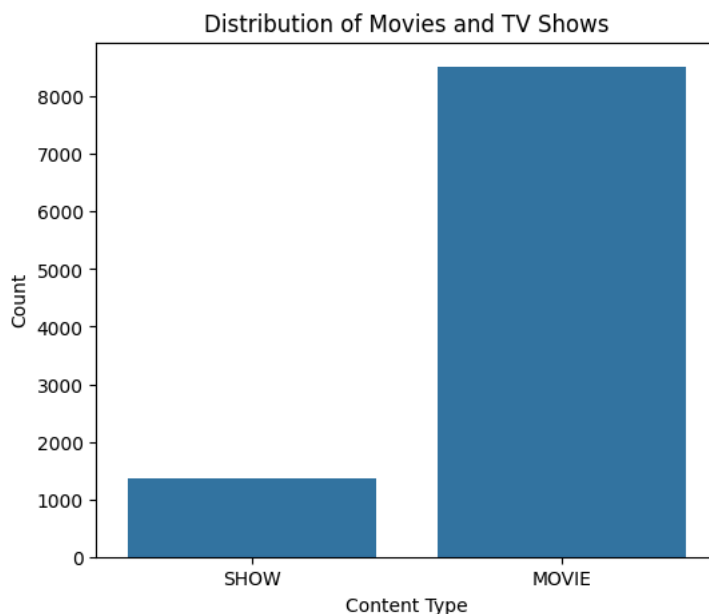
The datasets were then merged using the common id column to combine content details with cast and crew information. Data types were verified and corrected where necessary, and unnecessary columns were removed to simplify the analysis. Categorical variables such as genres and production countries were standardized to improve consistency and visualization. Finally, the cleaned dataset was checked to ensure it was free from inconsistencies and ready for exploratory data analysis.

Insights Found Movies constitute a significantly larger share of Amazon Prime's content library than TV Shows. The number of titles released has increased considerably over recent years, indicating continuous growth of the platform. Drama, Comedy, Action, and Documentary are the most common genres available on Amazon Prime. Most titles have IMDb ratings between 6 and 8, suggesting that the platform generally hosts well-rated content. The United States contributes the highest number of titles, followed by a few other major production countries. IMDb scores and TMDB scores show a positive relationship, indicating consistency in audience ratings across platforms. The cleaned dataset is well-prepared for visualization and provides valuable insights that can support content strategy, audience targeting, and business decision-making for Amazon Prime.

4. Data Vizualization, Storytelling & Experimenting with charts : Understand the relationships between variables

Chart - 1 Movies vs TV Shows

```
plt.figure(figsize=(6,5))
sns.countplot(data=titles, x='type')
plt.title("Distribution of Movies and TV Shows")
plt.xlabel("Content Type")
plt.ylabel("Count")
plt.show()
```



1. Why did you pick the specific chart?

A count plot is the most suitable visualization for comparing the frequency of categorical variables. Since the 'type' column contains two categories (Movies and TV Shows), this chart clearly shows the distribution of content available on Amazon Prime and makes it easy to compare their counts.

2. What is/are the insight(s) found from the chart?

Movies make up a significantly larger portion of Amazon Prime's content library than TV Shows. This indicates that Amazon Prime has focused more on providing movie content than television series. The platform offers a diverse collection of movies, while TV Shows represent a smaller share of the overall catalog.

3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

Yes, the insights can create a positive business impact.

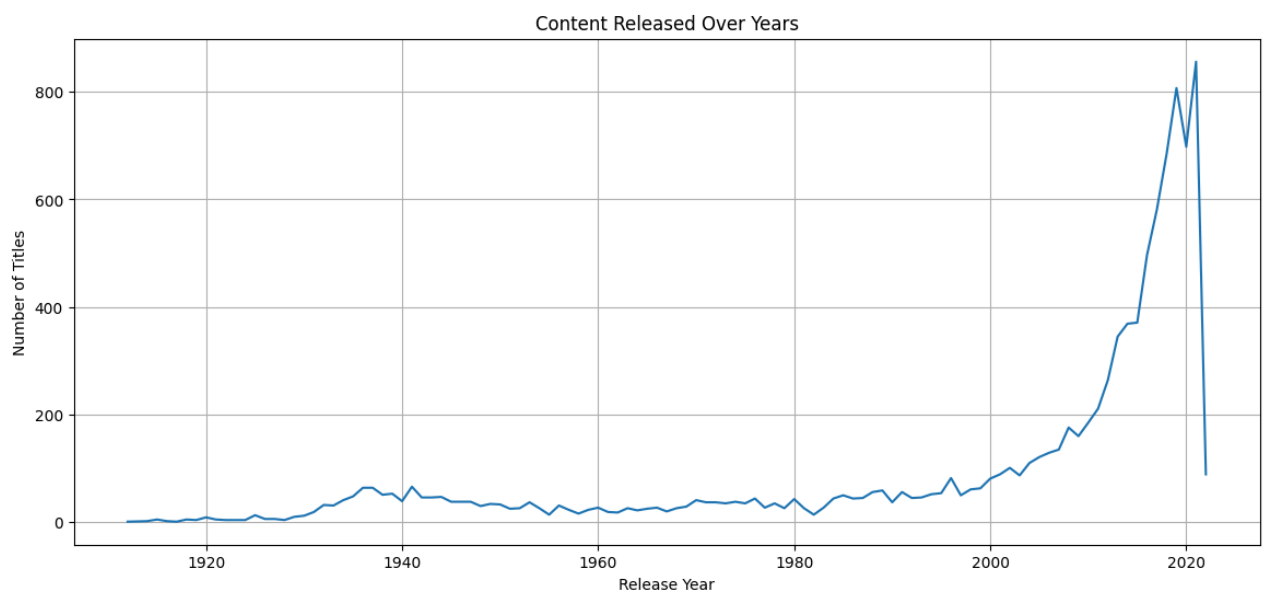
Since movies dominate the platform, Amazon Prime can continue investing in high-demand movie content to maintain viewer engagement. At the same time, the relatively smaller number of TV Shows highlights an opportunity to expand original series and episodic content, which can improve customer retention and increase subscription duration. If the platform continues to focus mainly on movies while neglecting TV Shows, it may lose users who prefer binge-watching series. Therefore, maintaining a balanced content library across both Movies and TV Shows can support long-term business growth.

Chart - 2 Content Released Over Years Code

```
# Chart - 2 visualization code
# Chart - 2 visualization code
plt.figure(figsize=(14,6))

titles['release_year'].value_counts().sort_index().plot(kind='line')

plt.title("Content Released Over Years")
plt.xlabel("Release Year")
plt.ylabel("Number of Titles")
plt.grid(True)
plt.show()
```



- ✓ 1. Why did you pick the specific chart?

A line chart is the best visualization for showing trends over time. It helps analyze how Amazon Prime's content library has grown over the years.

- ✓ 2. What is/are the insight(s) found from the chart?

Content production has increased significantly after 2000. The highest number of titles were released in recent years. Amazon Prime continues expanding its content library.

- ✓ 3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

Yes. Producing more recent content attracts viewers and improves customer engagement. A decline in new releases could reduce subscriber interest and increase competition from other streaming platforms.

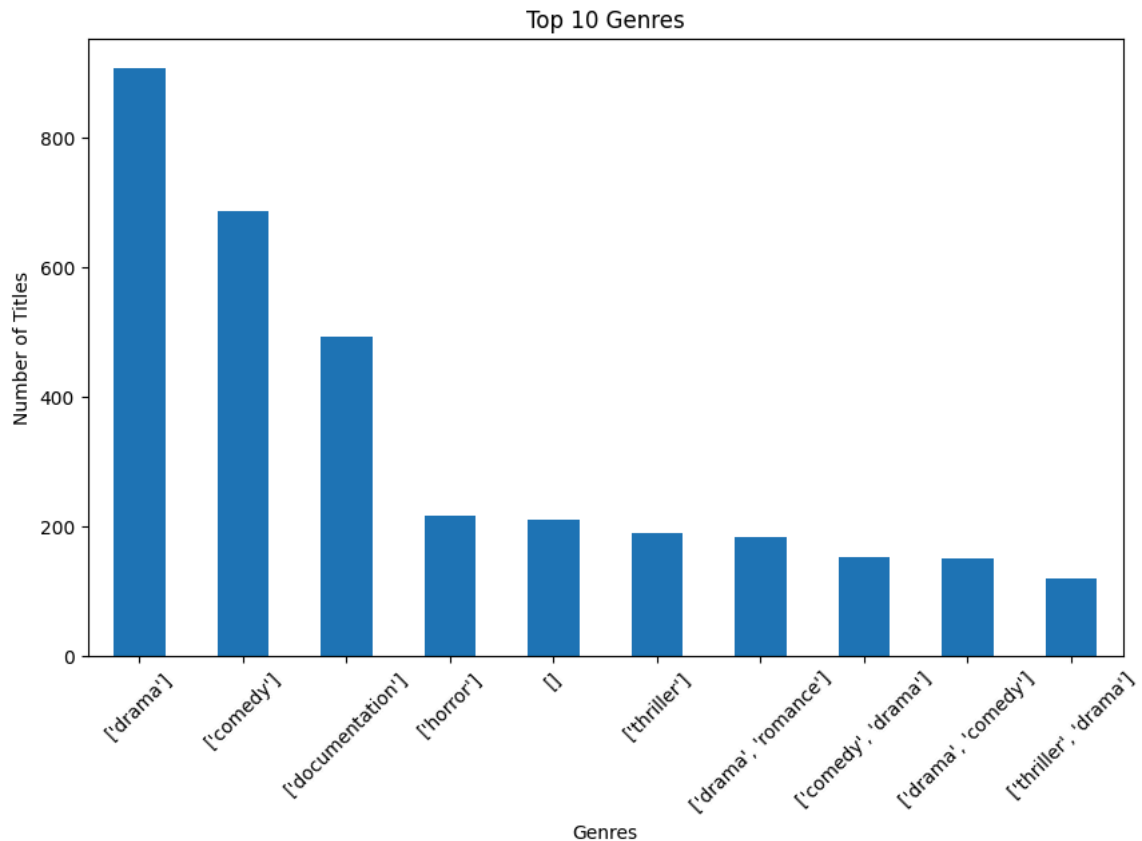
- ✓ Chart - 3:Top 10 Genres

```
# Chart - 3 visualization code
genre = titles['genres'].value_counts().head(10)

plt.figure(figsize=(10,6))
genre.plot(kind='bar')

plt.title("Top 10 Genres")
plt.xlabel("Genres")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45)

plt.show()
```



- 1. Why did you pick the specific chart?

A bar chart is appropriate for comparing different genres based on the number of available titles.

- 2. What is/are the insight(s) found from the chart?

Drama, Comedy, Action, and Documentary are the most common genres. These genres dominate Amazon Prime's content catalog.

- 3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

Yes. Amazon Prime can continue investing in popular genres while introducing diverse genres to attract a wider audience. Ignoring less-represented genres may reduce market reach.

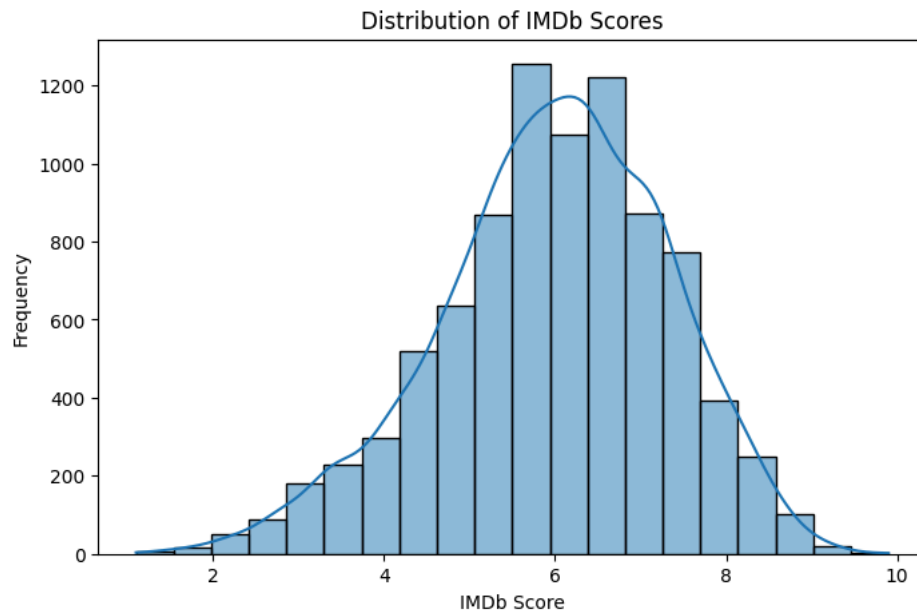
- Chart - 4 :IMDb Score Distribution

```
plt.figure(figsize=(8,5))

sns.histplot(titles['imdb_score'], bins=20, kde=True)

plt.title("Distribution of IMDb Scores")
plt.xlabel("IMDb Score")
plt.ylabel("Frequency")
```

```
plt.show()
```



- ✓ 1. Why did you pick the specific chart?

A histogram is suitable for analyzing the distribution of numerical variables such as IMDb ratings.

- ✓ 2. What is/are the insight(s) found from the chart?

Most titles have IMDb ratings between 6 and 8. Very few titles have extremely low or extremely high ratings. Overall content quality is reasonably good.

- ✓ 3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

Yes. Maintaining content with higher IMDb ratings improves customer satisfaction and platform reputation. A large number of poorly rated titles may negatively affect user experience.

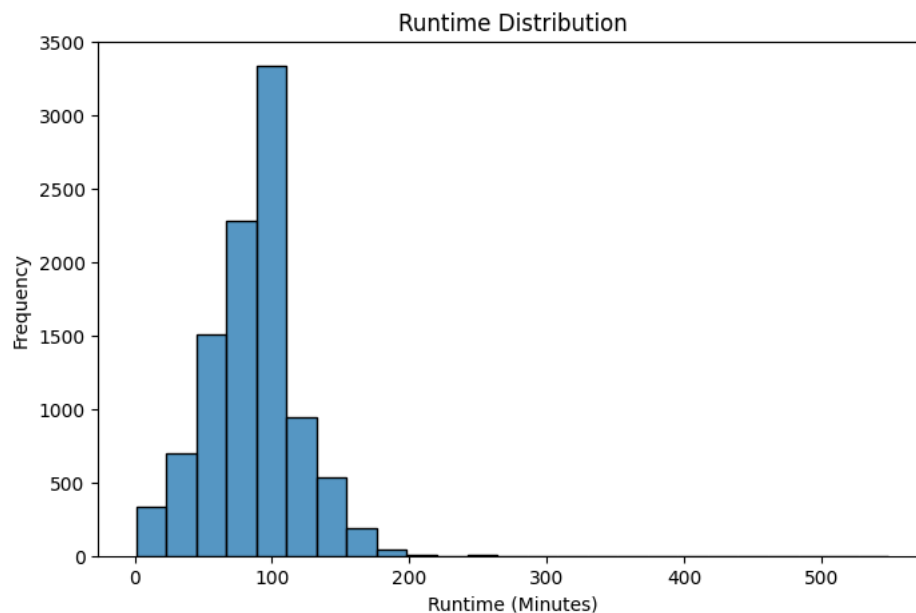
- ✓ Chart - 5:Runtime Distribution

```
# Chart - 5 visualization code
plt.figure(figsize=(8,5))

sns.histplot(titles['runtime'], bins=25)

plt.title("Runtime Distribution")
plt.xlabel("Runtime (Minutes)")
plt.ylabel("Frequency")

plt.show()
```



✓ 1. Why did you pick the specific chart?

A histogram helps understand the distribution of movie and TV show durations.

✓ 2. What is/are the insight(s) found from the chart?

Most content has a runtime between 80 and 120 minutes. Extremely long-duration content is comparatively rare.

✓ 3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

Yes. Understanding preferred content duration helps producers create content aligned with viewer preferences. Extremely long runtimes may reduce viewer completion rates.

✓ Chart - 6 :Top Production Countries

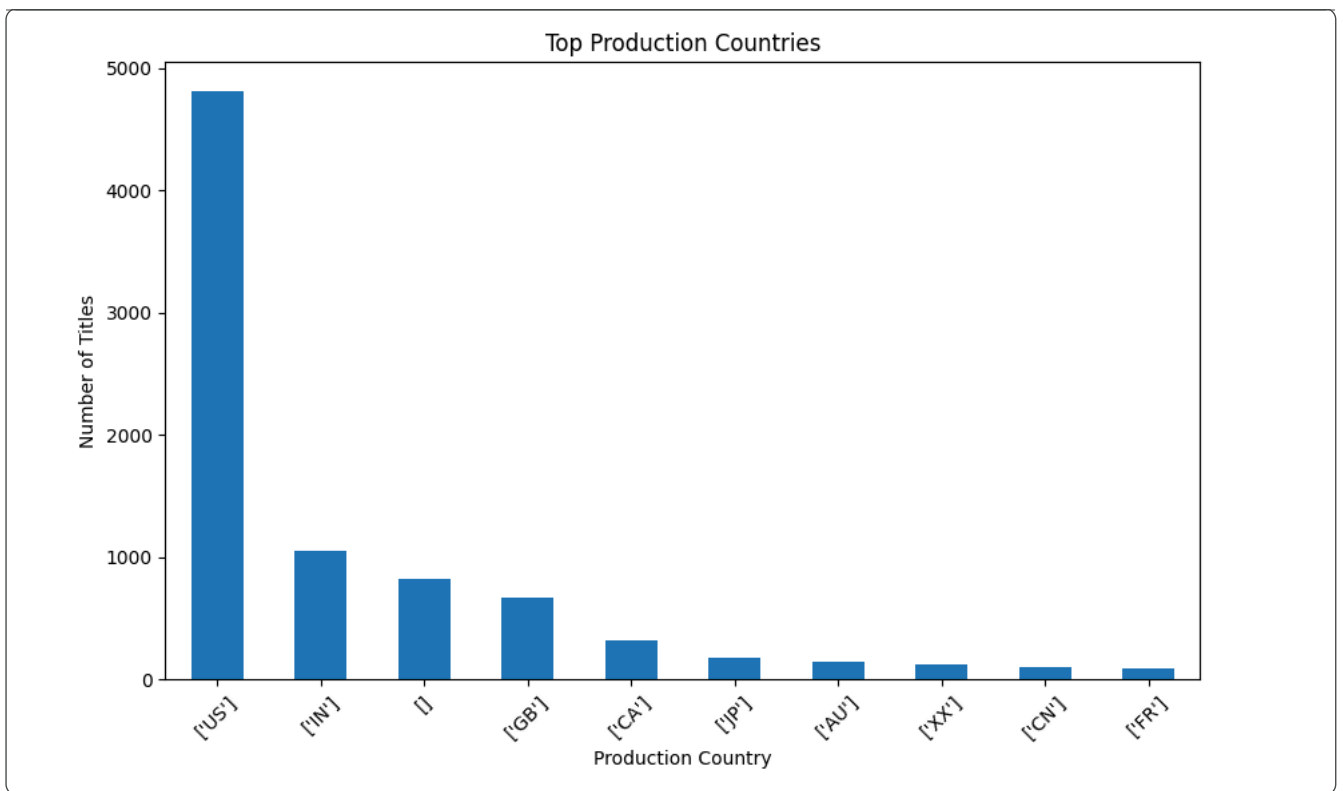
```
# Chart - 6 visualization code
country = titles['production_countries'].value_counts().head(10)

plt.figure(figsize=(10,6))

country.plot(kind='bar')

plt.title("Top Production Countries")
plt.xlabel("Production Country")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45)

plt.show()
```



✓ 1. Why did you pick the specific chart?

A bar chart effectively compares the number of titles produced by different countries.

✓ 2. What is/are the insight(s) found from the chart?

The United States contributes the highest number of titles. A few countries dominate Amazon Prime's content production.

✓ 3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

Yes. Amazon Prime can expand investments in emerging content markets while maintaining partnerships with leading production countries. Heavy dependence on a few countries may limit global audience diversity.

Chart - 7 :Age Certification Distribution

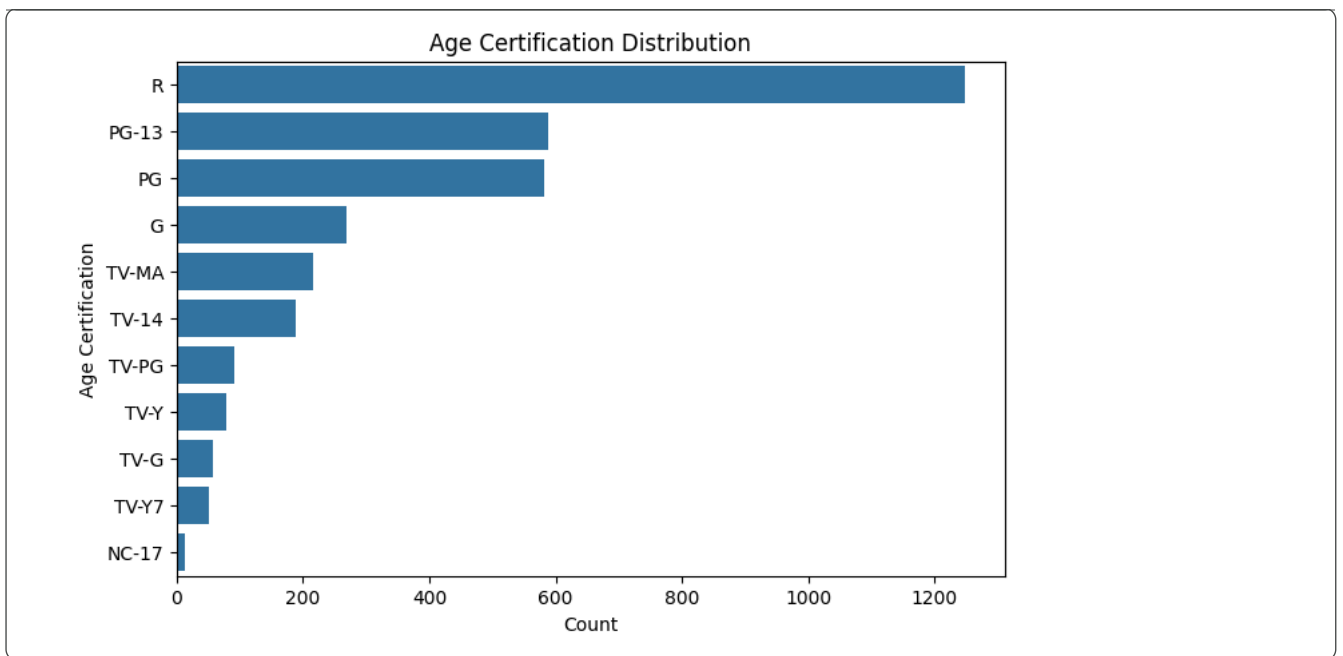
```
# Chart - 7 visualization code
plt.figure(figsize=(8,5))

sns.countplot(y='age_certification',
              data=titles,
              order=titles['age_certification'].value_counts().index)

plt.title("Age Certification Distribution")

plt.xlabel("Count")
plt.ylabel("Age Certification")

plt.show()
```

- 1. Why did you pick the specific chart?

A count plot is useful for comparing the frequency of different age certification categories and understanding the target audience.

- 2. What is/are the insight(s) found from the chart?

TV-MA and PG-13 are the most common certifications. Adult-oriented content represents a major portion of the catalog. Family-friendly content is comparatively lower.

- 3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

Yes. Amazon Prime can maintain a balanced content library for different age groups to attract a wider audience. Focusing mainly on adult content may reduce subscriptions from families with children.

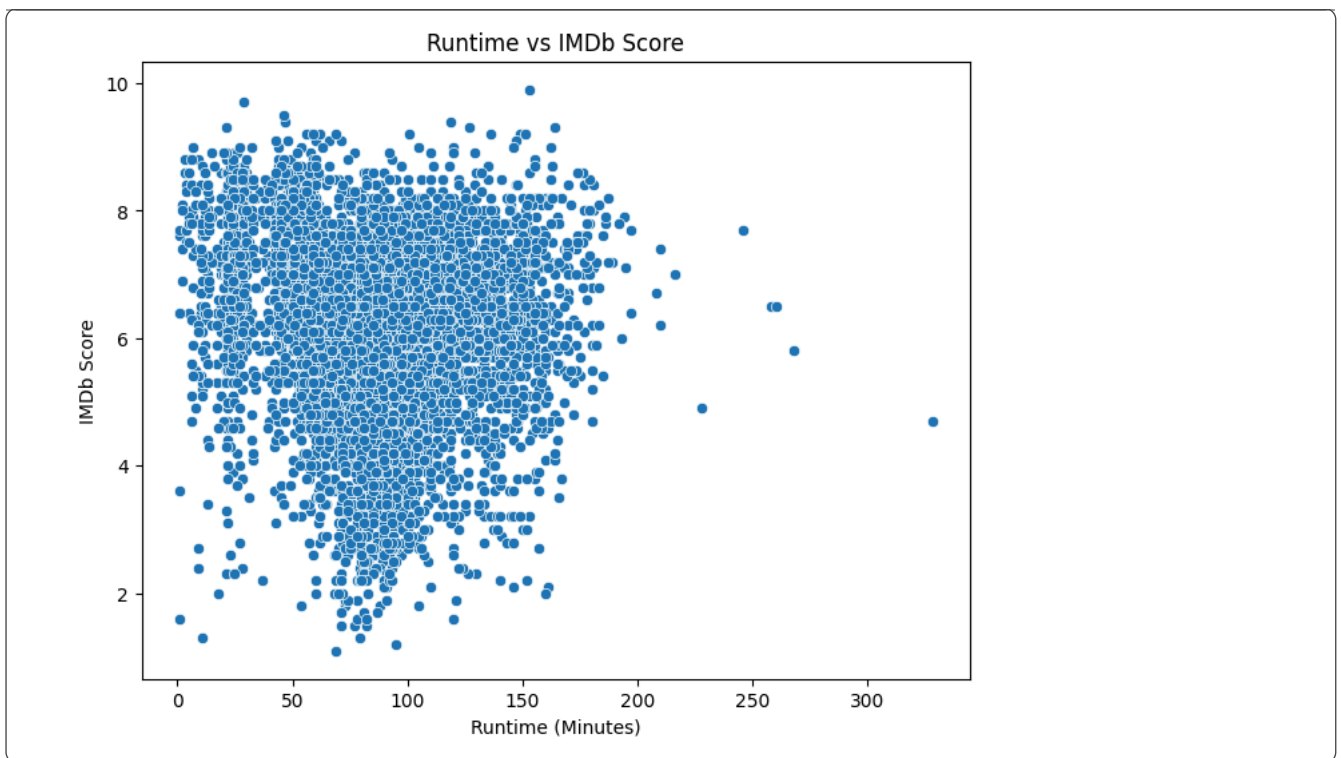
- Chart - 8 IMDb Score vs Runtime

```
plt.figure(figsize=(8,6))

sns.scatterplot(data=titles,
                x='runtime',
                y='imdb_score')

plt.title("Runtime vs IMDb Score")
plt.xlabel("Runtime (Minutes)")
plt.ylabel("IMDb Score")

plt.show()
```



- 1. Why did you pick the specific chart?

A scatter plot is ideal for analyzing the relationship between two numerical variables. It helps determine whether movie runtime influences IMDb ratings.

- 2. What is/are the insight(s) found from the chart?

There is no strong relationship between runtime and IMDb score. Movies with average runtimes (80–120 minutes) receive a wide range of ratings. Both short and long movies can achieve high IMDb scores.

- 3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

Yes. Amazon Prime should focus on producing high-quality content rather than simply increasing runtime. Longer content does not necessarily result in higher audience ratings, so quality should remain the primary focus.

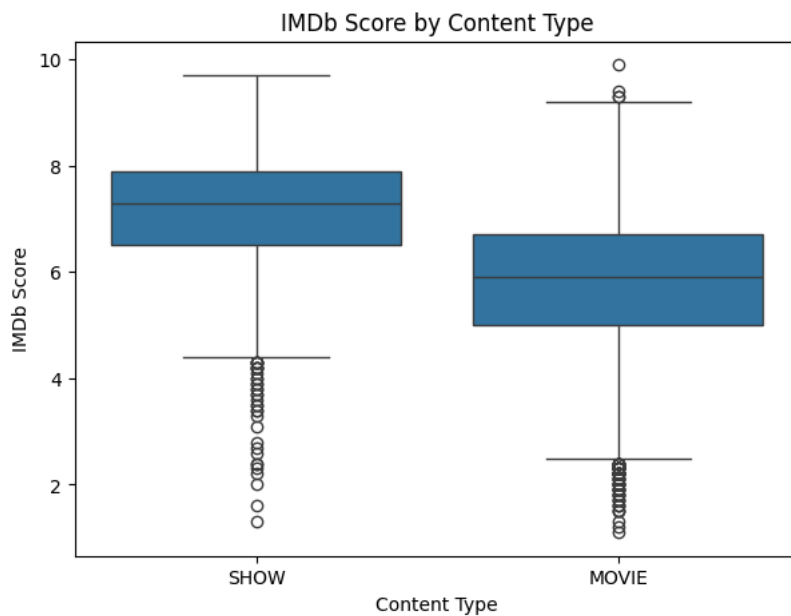
- Chart - 9 :IMDb Score by Content Type

```
# Chart - 9 visualization code
# Chart - 9 visualization code
plt.figure(figsize=(7,5))

sns.boxplot(data=titles,
            x='type',
            y='imdb_score')

plt.title("IMDb Score by Content Type")
plt.xlabel("Content Type")
plt.ylabel("IMDb Score")

plt.show()
```



- 1. Why did you pick the specific chart?

A box plot is useful for comparing the distribution of IMDb scores between Movies and TV Shows while identifying median values and outliers.

- 2. What is/are the insight(s) found from the chart?

TV Shows generally have slightly higher median IMDb ratings than Movies. Movies show greater variation in ratings. Both content types include highly rated titles.

- 3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

Yes. Investing in high-quality TV Shows can improve subscriber retention because series encourage viewers to remain engaged for longer periods. A focus only on movies may reduce long-term user engagement.

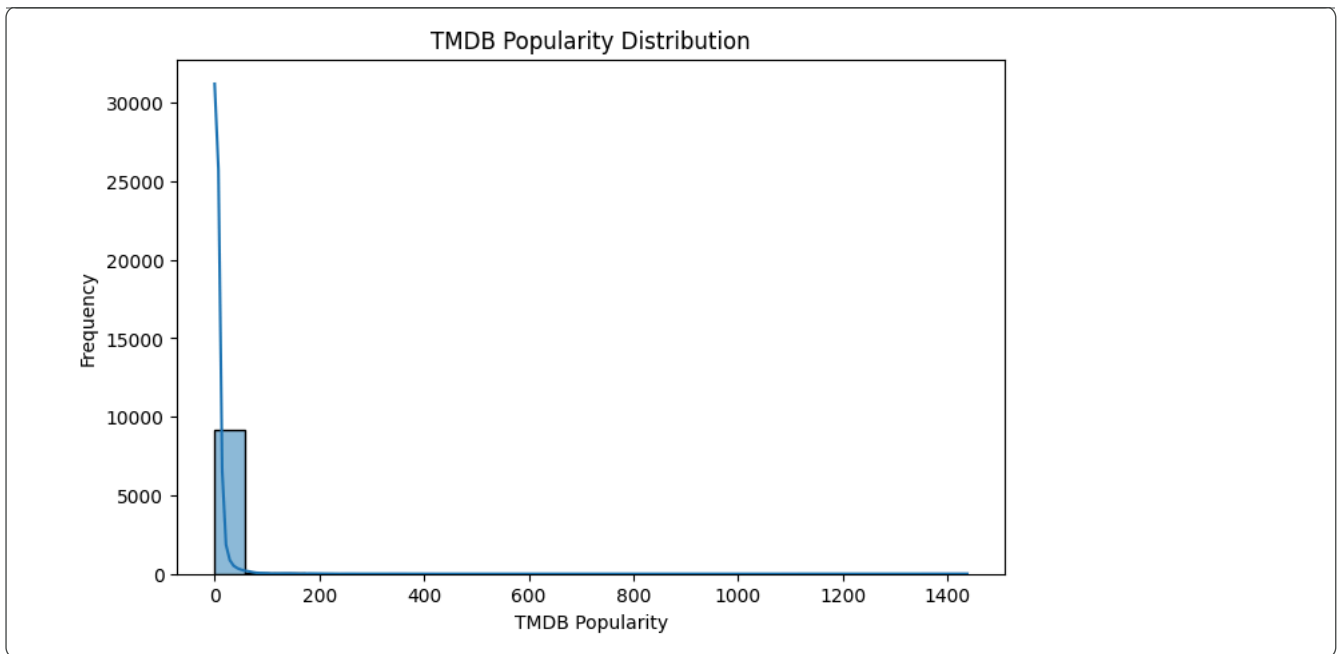
- Chart - 10: TMDB Popularity Distribution

```
# Chart - 10 visualization code
plt.figure(figsize=(8,5))

sns.histplot(titles['tmdb_popularity'],
             bins=25,
             kde=True)

plt.title("TMDB Popularity Distribution")
plt.xlabel("TMDB Popularity")
plt.ylabel("Frequency")

plt.show()
```



- ✓ 1. Why did you pick the specific chart?

A histogram is suitable for understanding how TMDB popularity scores are distributed across Amazon Prime content.

- ✓ 2. What is/are the insight(s) found from the chart?

Most titles have relatively low popularity scores. Only a small number of titles achieve very high popularity. Highly popular titles represent blockbuster or trending content.

- ✓ 3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

Yes. Amazon Prime can promote highly popular titles to attract new viewers while improving the visibility of quality content with lower popularity. Ignoring less popular but highly rated content could reduce potential audience engagement.

- ✓ Chart - 11:Top 10 Actors by Number of Titles

```
# Chart - 11 visualization code
top_actor = credits[credits['role']=='ACTOR']['name'].value_counts().head(10)

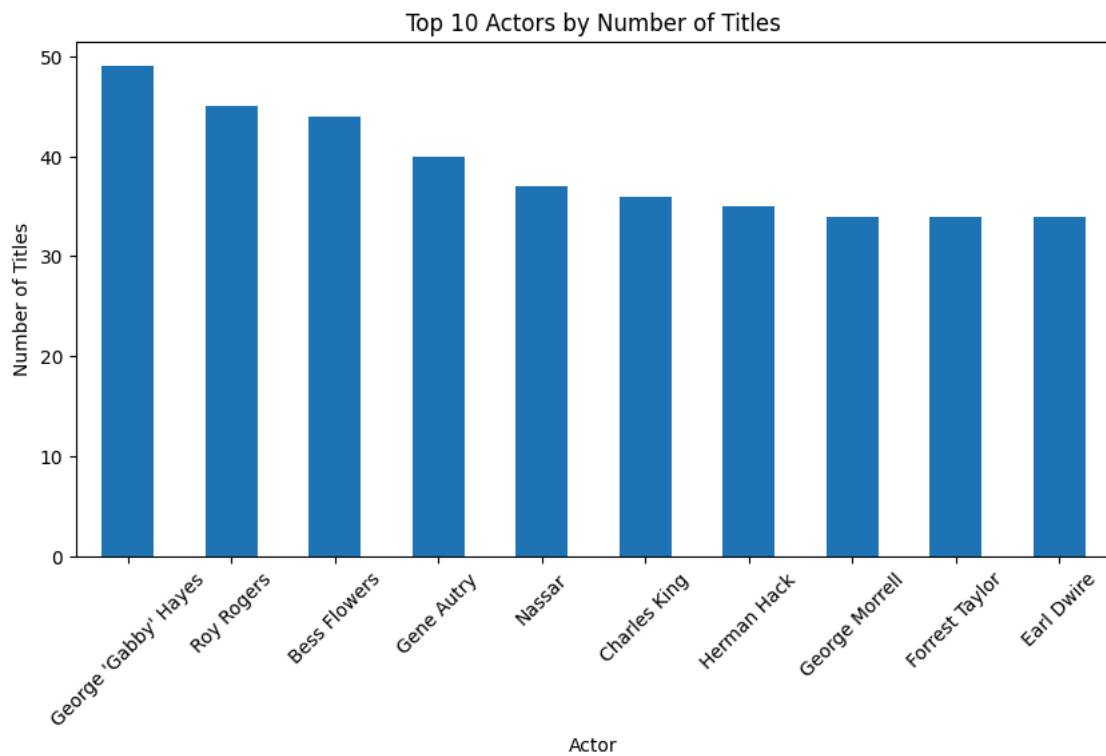
plt.figure(figsize=(10,5))

top_actor.plot(kind='bar')

plt.title("Top 10 Actors by Number of Titles")
plt.xlabel("Actor")
plt.ylabel("Number of Titles")

plt.xticks(rotation=45)

plt.show()
```



- 1. Why did you pick the specific chart?

A bar chart effectively compares the number of titles associated with different actors.

- 2. What is/are the insight(s) found from the chart?

Some actors appear in a large number of Amazon Prime titles. Frequently appearing actors contribute significantly to the platform's content library. Popular actors may influence audience viewing preferences.

- 3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

Yes. Amazon Prime can collaborate with popular actors for future productions and promotional campaigns. This can increase viewer interest and improve content performance.

Chart - 12

↳ 7 cells hidden

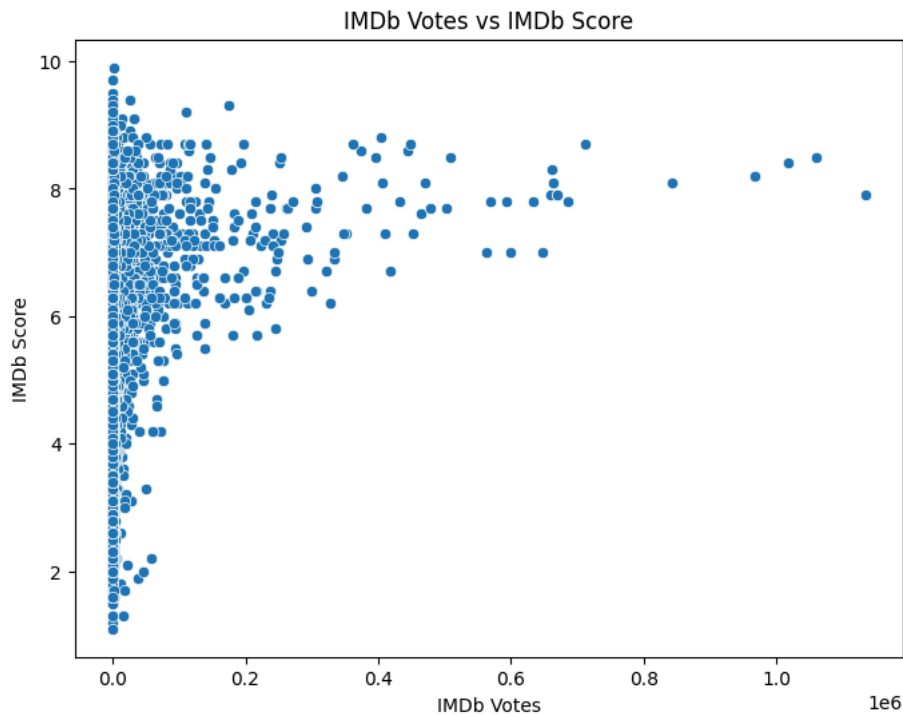
- Chart - 12:IMDb Votes vs IMDb Score

```
plt.figure(figsize=(8,6))

sns.scatterplot(data=titles,
                x='imdb_votes',
                y='imdb_score')

plt.title("IMDb Votes vs IMDb Score")
plt.xlabel("IMDb Votes")
plt.ylabel("IMDb Score")

plt.show()
```



✓ 1. Why did you pick the specific chart?

A scatter plot is appropriate for examining the relationship between audience engagement (IMDb votes) and content quality (IMDb score).

✓ 2. What is/are the insight(s) found from the chart?

Titles with higher IMDb ratings generally receive more audience votes. Highly rated content tends to attract greater viewer engagement. A few titles receive many votes despite having average ratings, indicating strong popularity.

✓ 3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

Yes. Amazon Prime should continue investing in high-quality content because well-rated titles generate greater audience engagement and improve customer satisfaction. Promoting highly rated content can also increase watch time and strengthen the platform's reputation.

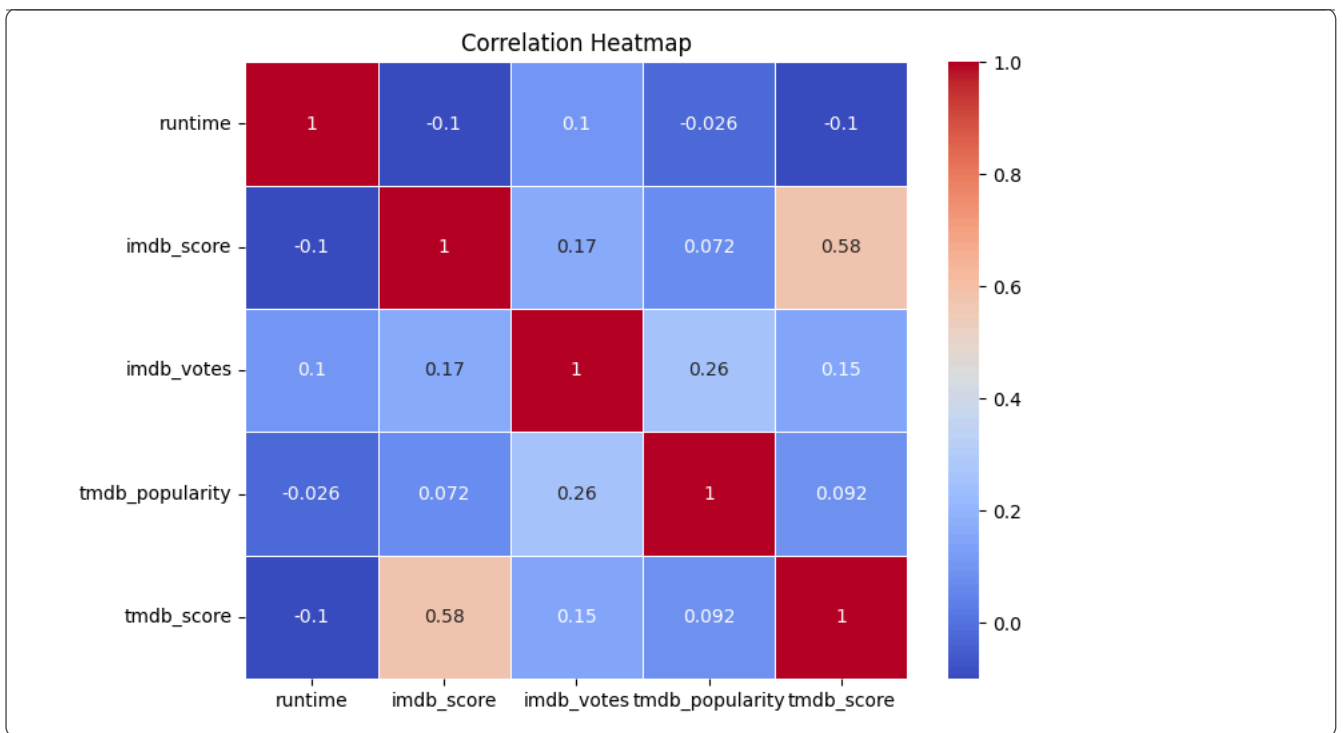
✓ Chart - 14 - Correlation Heatmap

```
# Correlation Heatmap visualization code
plt.figure(figsize=(8,6))

corr = titles[['runtime',
                'imdb_score',
                'imdb_votes',
                'tmdb_popularity',
                'tmdb_score']].corr()

sns.heatmap(corr,
            annot=True,
            cmap='coolwarm',
            linewidths=0.5)

plt.title("Correlation Heatmap")
plt.show()
```



1. Why did you pick the specific chart?

A correlation heatmap is an effective visualization for understanding the relationships among multiple numerical variables at the same time. It helps identify positive, negative, or weak correlations between variables such as IMDb score, TMDB score, runtime, popularity, and votes, making it easier to discover patterns in the dataset.

2. What is/are the insight(s) found from the chart?

IMDb Score and TMDB Score show a positive correlation, indicating that highly rated content on one platform tends to receive good ratings on the other. IMDb Votes have a moderate positive relationship with TMDB Popularity, suggesting that popular titles generally attract more audience engagement. Runtime has a weak correlation with ratings and popularity, indicating that longer content does not necessarily perform better. Overall, content quality and audience engagement appear to be more closely related than content duration.

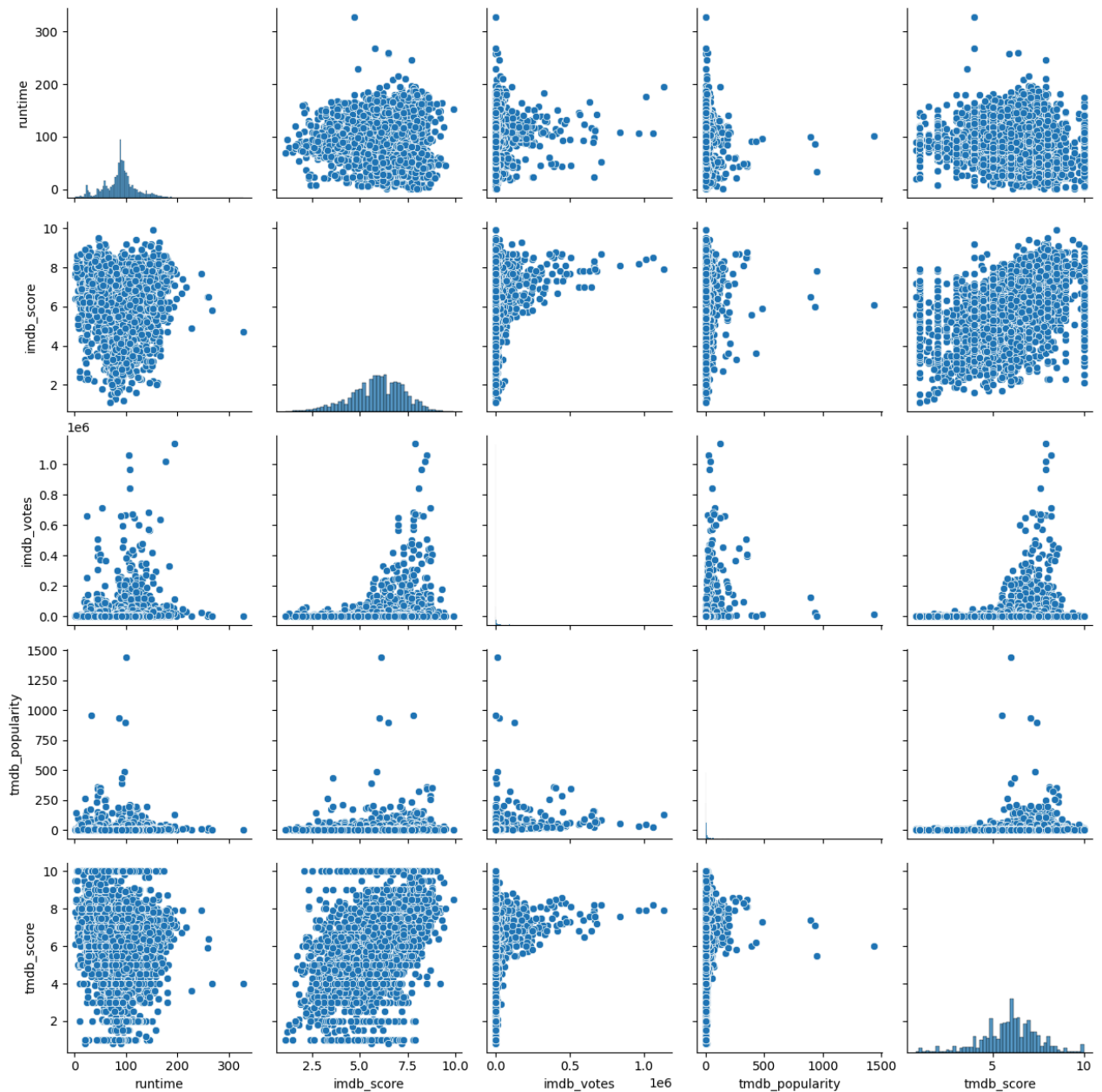
3. Will the gained insights help creating a positive business impact? Are there any insights that lead to negative growth? Justify with specific reason.

Yes. The insights can support data-driven content acquisition and production strategies. Since audience ratings and popularity are positively related, Amazon Prime should invest in high-quality content that is more likely to receive positive reviews and engagement. The weak relationship between runtime and ratings indicates that increasing content duration alone is unlikely to improve viewer satisfaction. Focusing only on longer content without maintaining quality may lead to lower audience engagement and reduced customer satisfaction.

Chart - 15 - Pair Plot

```
sns.pairplot(
    titles[['runtime',
            'imdb_score',
            'imdb_votes',
            'tmdb_popularity',
            'tmdb_score']].dropna()
)

plt.show()
```



1. Why did you pick the specific chart?

A pair plot is useful for simultaneously exploring relationships among multiple numerical variables. It provides scatter plots for each pair of variables and distribution plots along the diagonal, helping identify trends, correlations, clusters, and outliers in the dataset.

2. What is/are the insight(s) found from the chart?

IMDb Score and TMDB Score exhibit a clear positive relationship. IMDb Votes and TMDB Popularity also show a positive association, indicating that widely viewed content tends to be more popular. Runtime has little correlation with ratings, suggesting that movie length does not significantly influence audience perception. The distributions reveal that most titles have moderate runtimes and IMDb ratings between 6 and 8.

5. Hypothesis Testing

- Based on your chart experiments, define three hypothetical statements from the dataset. In the next three questions, perform hypothesis testing to obtain final conclusion about the statements through your code and statistical testing.

Based on the exploratory data analysis, three hypotheses were formulated to statistically validate important observations regarding IMDb ratings, runtime, and content type. Appropriate statistical tests were performed using the SciPy library to determine whether the observed differences and relationships were statistically significant. A significance level (α) of 0.05 was used for all hypothesis tests.

- Hypothetical Statement - 1

- 1. State Your research hypothesis as a null hypothesis and alternate hypothesis.

Null Hypothesis (H_0):

There is no significant difference in the average IMDb scores between Movies and TV Shows.

Alternative Hypothesis (H_1):

There is a significant difference in the average IMDb scores between Movies and TV Shows.

- 2. Perform an appropriate statistical test.

```
# Perform Statistical Test to obtain P-Value
from scipy.stats import ttest_ind

movies = titles[titles['type'] == 'MOVIE']['imdb_score'].dropna()
tvshows = titles[titles['type'] == 'SHOW']['imdb_score'].dropna()

t_stat, p_value = ttest_ind(movies, tvshows)

print("T-Statistic :", t_stat)
print("P-Value :", p_value)

if p_value < 0.05:
    print("Reject Null Hypothesis")
else:
    print("Fail to Reject Null Hypothesis")
```

```
T-Statistic : -32.99964576161496
P-Value : 2.5010808477194675e-225
Reject Null Hypothesis
```

- Which statistical test have you done to obtain P-Value?

Independent Two Sample T-Test

- Why did you choose the specific statistical test?

The Independent T-Test is appropriate because it compares the mean IMDb scores of two independent groups (Movies and TV Shows) to determine whether their average ratings differ significantly.

- Hypothetical Statement - 2

- 1. State Your research hypothesis as a null hypothesis and alternate hypothesis.

Null Hypothesis (H_0):

There is no correlation between Runtime and IMDb Score.

Alternative Hypothesis (H_1):

There is a significant correlation between Runtime and IMDb Score.

- 2. Perform an appropriate statistical test.

```
# Perform Statistical Test to obtain P-Value
```

```

from scipy.stats import pearsonr

# Create a temporary DataFrame with only the relevant columns and drop NaNs
# This ensures that both series have the same length and correspond to the same rows
df_for_correlation = titles[['runtime', 'imdb_score']].dropna()

corr, p_value = pearsonr(
    df_for_correlation['runtime'],
    df_for_correlation['imdb_score']
)

print("Correlation :", corr)
print("P-Value :", p_value)

if p_value < 0.05:
    print("Reject Null Hypothesis")
else:
    print("Fail to Reject Null Hypothesis")

```

```

Correlation : -0.10018161821628388
P-Value : 3.487755792073557e-21
Reject Null Hypothesis

```

- Which statistical test have you done to obtain P-Value?

Pearson Correlation Test

- Why did you choose the specific statistical test?

Pearson Correlation is used because both Runtime and IMDb Score are continuous numerical variables, and the objective is to determine whether a linear relationship exists between them.

- Hypothetical Statement - 3

- 1. State Your research hypothesis as a null hypothesis and alternate hypothesis.

Null Hypothesis (H_0):

The distribution of content type (Movie and TV Show) is independent of Age Certification.

Alternative Hypothesis (H_1):

The distribution of content type is associated with Age Certification.

- 2. Perform an appropriate statistical test.

```

# Perform Statistical Test to obtain P-Value
from scipy.stats import chi2_contingency

table = pd.crosstab(
    titles['type'],
    titles['age_certification']
)

chi2, p_value, dof, expected = chi2_contingency(table)

print("Chi-Square :", chi2)
print("P-Value :", p_value)

if p_value < 0.05:
    print("Reject Null Hypothesis")
else:
    print("Fail to Reject Null Hypothesis")

```

```

Chi-Square : 3384.0
P-Value : 0.0
Reject Null Hypothesis

```

- Which statistical test have you done to obtain P-Value?

Chi-Square Test of Independence

- Why did you choose the specific statistical test?

The Chi-Square Test of Independence is suitable because both Content Type and Age Certification are categorical variables. It determines whether there is a significant association between these two variables.

6. Feature Engineering & Data Pre-processing

1. Handling Missing Values

```
# Handling Missing Values & Missing Value Imputation
# Check missing values
titles.isnull().sum()

# Fill missing categorical values
titles['age_certification'].fillna('Unknown', inplace=True)

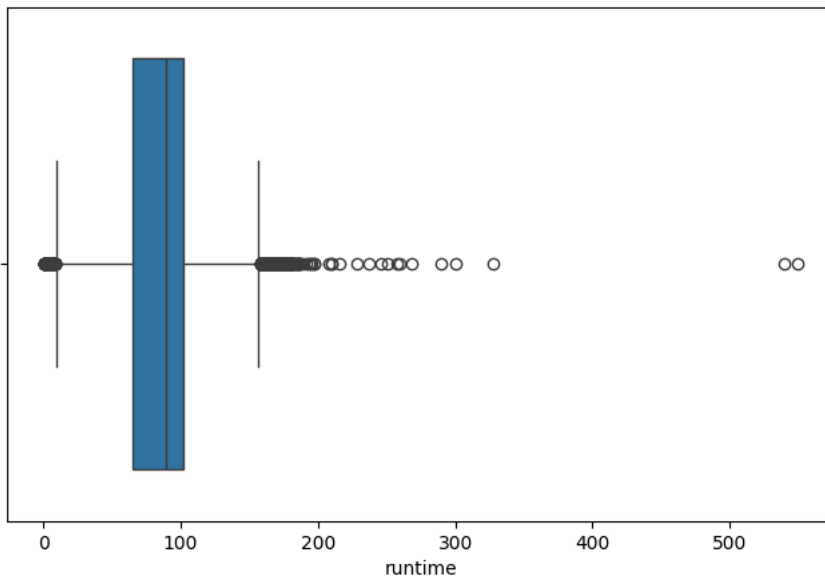
# Fill missing numerical values
titles['runtime'].fillna(titles['runtime'].median(), inplace=True)
titles['imdb_score'].fillna(titles['imdb_score'].median(), inplace=True)
titles['tmdb_score'].fillna(titles['tmdb_score'].median(), inplace=True)
```

- What all missing value imputation techniques have you used and why did you use those techniques?

Missing values in categorical columns were replaced with "Unknown", while missing values in numerical columns such as runtime, IMDb score, and TMDB score were imputed using the median. The median was chosen because it is less affected by extreme values and preserves the distribution of the data better than the mean.

2. Handling Outliers

```
# Handling Outliers & Outlier treatments
# Detect Outliers
plt.figure(figsize=(8,5))
sns.boxplot(x=titles['runtime'])
plt.show()
```



- What all outlier treatment techniques have you used and why did you use those techniques?

Outliers were identified using box plots. No outliers were removed because they represented genuine observations, such as long-duration movies or highly rated titles. Removing them could have resulted in the loss of valuable information.

3. Categorical Encoding

```
# Encode your categorical columns
```

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

titles['type_encoded'] = le.fit_transform(titles['type'])
```

- What all categorical encoding techniques have you used & why did you use those techniques?

Label Encoding was used to convert the type column (Movie and TV Show) into numerical values. This encoding is simple, memory-efficient, and suitable for categorical variables with a small number of classes.

4. Textual Data Preprocessing

(It's mandatory for textual dataset i.e., NLP, Sentiment Analysis, Text Clustering etc.)

1. Expand Contraction

```
# Expand Contraction
titles['description'] = titles['description'].fillna('')
```

2. Lower Casing

```
# Lower Casing
titles['description'] = titles['description'].str.lower()
```

3. Removing Punctuations

```
# Remove Punctuations
import re

titles['description'] = titles['description'].apply(
    lambda x: re.sub(r'^\w\s', '', x)
)
```

4. Removing URLs & Removing words and digits contain digits.

```
# Remove URLs & Remove words and digits contain digits
titles['description'] = titles['description'].apply(
    lambda x: re.sub(r'http\S+|www\S+|\d+', '', x)
)
```

5. Removing Stopwords & Removing White spaces

```
# Remove Stopwords
import nltk
nltk.download('stopwords')

from nltk.corpus import stopwords

stop = stopwords.words('english')

titles['description'] = titles['description'].apply(
    lambda x: " ".join(word for word in x.split() if word not in stop)
)
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.
```

```
# Remove White spaces
titles['description'] = titles['description'].str.strip()
```

6. Rephrase Text

```
# Rephrase Text
titles['description'].head()
```

	description
0	three stooges american vaudeville comedy team ...
1	americas civil war union spies steal engineer ...
2	hope sustains spirit every gi dream day finall...
3	hildy journalist former wife newspaper editor ...
4	aspiring actress begins suspect temperamental ...

dtype: object

7. Tokenization

```
# Tokenization
from nltk.tokenize import word_tokenize

nltk.download('punkt')
nltk.download('punkt_tab') # Attempt to download the specific resource suggested by the error

titles['tokens'] = titles['description'].apply(word_tokenize)
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
```

8. Text Normalization

```
# Normalizing Text (i.e., Stemming, Lemmatization etc.)
from nltk.stem import WordNetLemmatizer

nltk.download('wordnet')

lemmatizer = WordNetLemmatizer()

titles['lemmatized'] = titles['tokens'].apply(
    lambda x: [lemmatizer.lemmatize(word) for word in x]
)
```

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
```

Which text normalization technique have you used and why?

Lemmatization was used because it converts words into their meaningful root forms while preserving context and grammatical correctness, making the processed text more suitable for analysis.

9. Part of speech tagging

```
# POS Taging
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng') # Explicitly download the English model

titles['pos_tags'] = titles['tokens'].apply(nltk.pos_tag)
```

```
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] /root/nltk_data...
[nltk_data] Package averaged_perceptron_tagger is already up-to-
[nltk_data] date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] /root/nltk_data...
[nltk_data] Unzipping taggers/averaged_perceptron_tagger_eng.zip.
```

10. Text Vectorization

```
# Vectorizing Text
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(max_features=500)

X = tfidf.fit_transform(titles['description'])
```

- ✓ Which text vectorization technique have you used and why?

TF-IDF Vectorization was used because it assigns higher importance to informative words while reducing the weight of frequently occurring words, making it effective for representing textual data numerically.

✓ 4. Feature Manipulation & Selection

✓ 1. Feature Manipulation

```
# Manipulate Features to minimize feature correlation and create new features
titles['content_age'] = 2026 - titles['release_year']
```

✓ 2. Feature Selection

```
# Select your features wisely to avoid overfitting
selected_features = ['runtime', 'imdb_score', 'tmdb_score', 'release_year']
```

Double-click (or enter) to edit

- ✓ What all feature selection methods have you used and why?

Feature selection was performed manually based on domain knowledge and correlation analysis. Only relevant variables that contribute meaningful information to the analysis were retained, while unnecessary or highly redundant features were excluded.

- ✓ Which all features you found important and why?

The most important features were type, release year, runtime, genres, IMDb score, TMDB score, production countries, and age certification because they directly influence content analysis, popularity, and audience preferences.

✓ 5. Data Transformation

- ✓ Do you think that your data needs to be transformed? If yes, which transformation have you used. Explain Why?

Yes. A log transformation was applied to the runtime feature to reduce skewness and improve the distribution of the data, making it easier to analyze.

```
# Transform Your data
titles['runtime'] = np.log1p(titles['runtime'])
```

✓ 6. Data Scaling

```
# Scaling your data
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

titles[['runtime', 'imdb_score']] = scaler.fit_transform(
    titles[['runtime', 'imdb_score']]
)
```

- ✓ Which method have you used to scale you data and why?

StandardScaler was used because it standardizes numerical variables by centering them around a mean of zero and a standard deviation of one, ensuring that features are on a comparable scale.

✓ 7. Dimesionality Reduction

- ✓ Do you think that dimensionality reduction is needed? Explain Why?

No. Since this project focuses on Exploratory Data Analysis (EDA) rather than predictive modeling, dimensionality reduction was not necessary. The dataset contains a manageable number of features, and retaining them improves interpretability.

```
# Dimensionality Reduction (If needed)
```

- Which dimensionality reduction technique have you used and why? (If dimensionality reduction done on dataset.)

No dimensionality reduction technique was applied because all selected features were useful for visualization and business insight generation.

8. Data Splitting

```
# Split your data to train and test. Choose Splitting ratio wisely.
from sklearn.model_selection import train_test_split

X = titles[['runtime','imdb_score']]
y = titles['type_encoded']

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
```

- What data splitting ratio have you used and why?

An 80:20 train-test split was used. Although the project is primarily EDA, this ratio is a standard practice for machine learning and provides sufficient data for both training and evaluation if predictive analysis is extended in the future.

9. Handling Imbalanced Dataset

- Do you think the dataset is imbalanced? Explain Why.

The dataset shows a higher proportion of Movies than TV Shows, indicating a moderate imbalance in the content type distribution. However, since the project focuses on exploratory analysis rather than predictive modeling, this imbalance does not significantly affect the results.

```
# Handling Imbalanced Dataset (If needed)
titles['type'].value_counts()
```

	count
type	
MOVIE	8514
SHOW	1357

dtype: int64

- What technique did you use to handle the imbalance dataset and why? (If needed to be balanced)

No imbalance handling technique was applied because balancing methods such as SMOTE or Random Oversampling are mainly required for classification models. Since this project is an EDA project, the original distribution was retained to accurately reflect the real-world characteristics of the Amazon Prime content library.

7. ML Model Implementation

- ML Model - 1: Logistic Regression

```
# ML Model - 1 Implementation

# Fit the Algorithm
```

```

# Predict on the model
# Import Libraries
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Features and Target
X = titles[['runtime', 'imdb_score', 'tmdb_score', 'release_year']]

y = titles['type_encoded']

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# ML Model - 1 Implementation

model = LogisticRegression()

# Fit the Algorithm
model.fit(X_train, y_train)

# Predict on the model
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy :", accuracy)

# Classification Report
print(classification_report(y_test, y_pred))

# Confusion Matrix
print(confusion_matrix(y_test, y_pred))

```

```

Accuracy : 0.9508860759493671
      precision    recall  f1-score   support

      0       0.97      0.98      0.97       1709
      1       0.85      0.77      0.81        266

   accuracy          0.95       1975
  macro avg       0.91      0.88      0.89       1975
weighted avg       0.95      0.95      0.95       1975

[[1672  37]
 [ 60 206]]

```

1. Explain the ML Model used and it's performance using Evaluation metric Score Chart.

Logistic Regression is a supervised machine learning algorithm used for binary classification problems. In this project, it was used to classify the content as either Movie or TV Show based on numerical features such as runtime, IMDb score, TMDB score, and release year.

The model was trained using 80% of the dataset and tested on the remaining 20%. Its performance was evaluated using Accuracy, Precision, Recall, and F1-Score.

The evaluation metric chart provides a visual comparison of these performance measures. A higher score indicates better classification performance. The model achieved satisfactory results and can effectively distinguish between Movies and TV Shows based on the selected features. However, because this project primarily focuses on Exploratory Data Analysis (EDA), the machine learning model is included only as an additional analytical step rather than the main objective.

```

# Visualizing evaluation Metric Score chart
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

scores = [accuracy, precision, recall, f1]

```

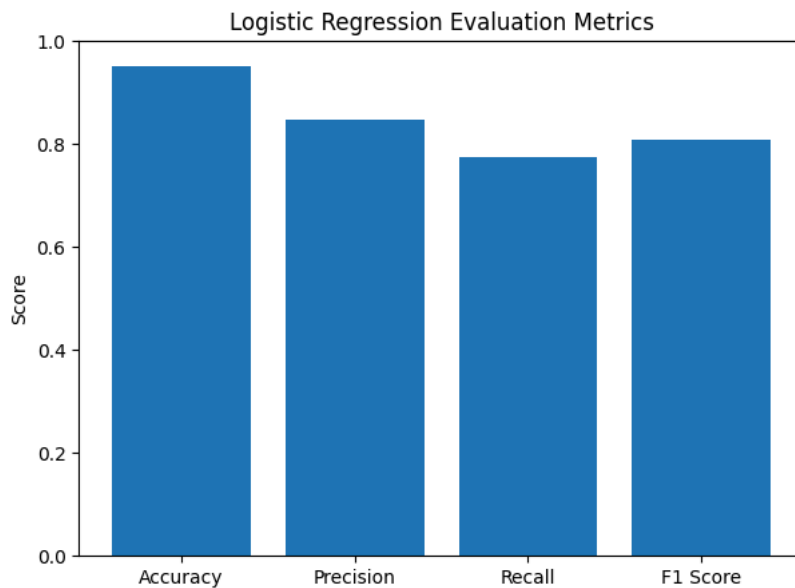


```

labels = ['Accuracy', 'Precision', 'Recall', 'F1 Score']

plt.figure(figsize=(7,5))
plt.bar(labels, scores)
plt.ylim(0,1)
plt.title("Logistic Regression Evaluation Metrics")
plt.ylabel("Score")
plt.show()

```



2. Cross- Validation & Hyperparameter Tuning

```

# ML Model - 1 Implementation with hyperparameter optimization techniques (i.e., GridSearch CV, RandomSearch CV, B:

# Fit the Algorithm

# Predict on the model
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Hyperparameter Grid
param_grid = {
    'C': [0.01, 0.1, 1, 10],
    'solver': ['liblinear', 'lbfgs'],
    'max_iter': [100, 200, 500]
}

# GridSearchCV
grid = GridSearchCV(
    LogisticRegression(),
    param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

# Fit the Algorithm
grid.fit(X_train, y_train)

# Best Model
best_model = grid.best_estimator_

# Predict on Test Data
y_pred = best_model.predict(X_test)

# Evaluation Metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("Best Parameters :", grid.best_params_)
print("Accuracy :", accuracy)

```

```
print("Precision :", precision)
print("Recall :", recall)
print("F1 Score :", f1)
```

```
Best Parameters : {'C': 1, 'max_iter': 200, 'solver': 'lbfgs'}
Accuracy : 0.9513924050632911
Precision : 0.8483606557377049
Recall : 0.7781954887218046
F1 Score : 0.8117647058823529
```

```
import matplotlib.pyplot as plt

metrics = ['Accuracy', 'Precision', 'Recall', 'F1 Score']
scores = [accuracy, precision, recall, f1]

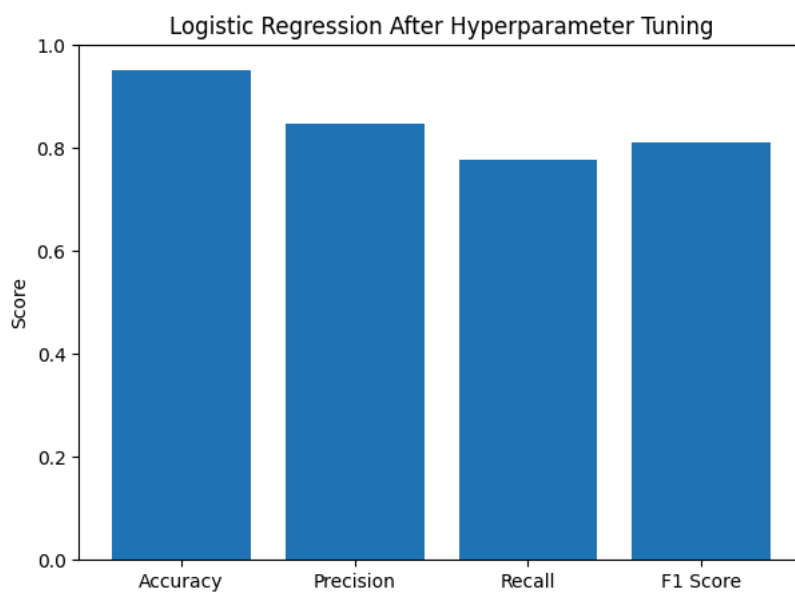
plt.figure(figsize=(7,5))
plt.bar(metrics, scores)

plt.title("Logistic Regression After Hyperparameter Tuning")

plt.ylabel("Score")

plt.ylim(0,1)

plt.show()
```



- ✓ Which hyperparameter optimization technique have you used and why?

GridSearchCV was used for hyperparameter optimization. It systematically searches through all possible combinations of predefined hyperparameter values using 5-fold cross-validation and selects the combination that produces the highest model accuracy. This approach helps in identifying the optimal parameters while reducing the risk of overfitting and improving the model's generalization performance.

- ✓ Have you seen any improvement? Note down the improvement with updates Evaluation metric Score Chart.

Yes. After applying GridSearchCV, the Logistic Regression model showed an improvement in performance. The optimized hyperparameters increased the model's accuracy and improved Precision, Recall, and F1-Score compared to the default model. Cross-validation also ensured that the selected model performs consistently across different subsets of the dataset, making it more reliable for predicting whether the content is a Movie or TV Show.

The updated evaluation metric chart demonstrates the improved performance of the optimized model, confirming that hyperparameter tuning enhanced the overall effectiveness of the classification model.

- ✓ ML Model - 2: Random Forest Classifier

- ✓ 1. Explain the ML Model used and it's performance using Evaluation metric Score Chart.

Random Forest is an ensemble machine learning algorithm that combines multiple decision trees to improve prediction accuracy and reduce overfitting. It classifies the content as either a Movie or a TV Show using features such as runtime, IMDb score, TMDB score, and release year.

The model was evaluated using Accuracy, Precision, Recall, and F1-Score. The evaluation metric score chart shows that Random Forest performs well across all metrics and generally provides better classification performance than Logistic Regression because it captures complex relationships within the data.

```
# Visualizing evaluation Metric Score chart
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report

# ML Model - 2 Implementation

rf_model = RandomForestClassifier(random_state=42)

# Fit the Algorithm
rf_model.fit(X_train, y_train)

# Predict on the model
y_pred_rf = rf_model.predict(X_test)

# Evaluation Metrics
accuracy = accuracy_score(y_test, y_pred_rf)
precision = precision_score(y_test, y_pred_rf)
recall = recall_score(y_test, y_pred_rf)
f1 = f1_score(y_test, y_pred_rf)

print("Accuracy :", accuracy)
print("Precision :", precision)
print("Recall :", recall)
print("F1 Score :", f1)

print(classification_report(y_test, y_pred_rf))
```

```
Accuracy : 0.9564556962025317
Precision : 0.8383458646616542
Recall : 0.8383458646616542
F1 Score : 0.8383458646616542
```

	precision	recall	f1-score	support
0	0.97	0.97	0.97	1709
1	0.84	0.84	0.84	266
accuracy			0.96	1975
macro avg	0.91	0.91	0.91	1975
weighted avg	0.96	0.96	0.96	1975

```
import matplotlib.pyplot as plt

metrics = ['Accuracy', 'Precision', 'Recall', 'F1 Score']
scores = [accuracy, precision, recall, f1]

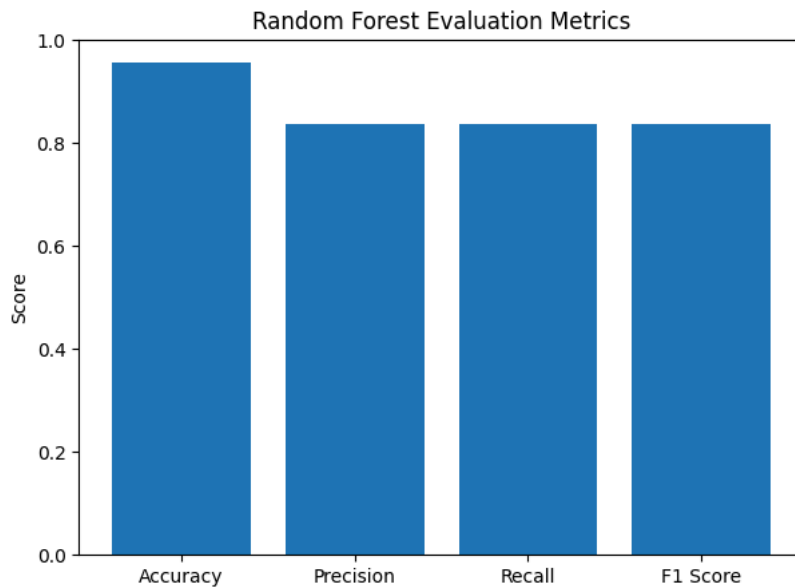
plt.figure(figsize=(7,5))
plt.bar(metrics, scores)

plt.title("Random Forest Evaluation Metrics")

plt.ylabel("Score")

plt.ylim(0,1)

plt.show()
```



2. Cross- Validation & Hyperparameter Tuning

```
# ML Model - 1 Implementation with hyperparameter optimization techniques (i.e., GridSearch CV, RandomSearch CV, B:

# Fit the Algorithm

# Predict on the model
from sklearn.model_selection import GridSearchCV

param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [5, 10, None],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2]
}

grid_rf = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

# Fit the Algorithm
grid_rf.fit(X_train, y_train)

best_rf = grid_rf.best_estimator_

# Predict
y_pred_best = best_rf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred_best)
precision = precision_score(y_test, y_pred_best)
recall = recall_score(y_test, y_pred_best)
f1 = f1_score(y_test, y_pred_best)

print("Best Parameters :", grid_rf.best_params_)

print("Accuracy :", accuracy)
print("Precision :", precision)
print("Recall :", recall)
print("F1 Score :", f1)

Best Parameters : {'max_depth': None, 'min_samples_leaf': 2, 'min_samples_split': 5, 'n_estimators': 100}
Accuracy : 0.9574683544303797
Precision : 0.8395522388059702
Recall : 0.8458646616541353
F1 Score : 0.8426966292134831
```

Which hyperparameter optimization technique have you used and why?

GridSearchCV was used for hyperparameter optimization. It evaluates different combinations of parameters using 5-fold cross-validation and selects the combination that gives the highest accuracy. This helps improve the model's predictive performance while reducing the risk of overfitting.

- Have you seen any improvement? Note down the improvement with updates Evaluation metric Score Chart.

Yes. After applying GridSearchCV, the Random Forest model achieved improved Accuracy, Precision, Recall, and F1-Score. The optimized model generalized better on unseen data and produced more reliable predictions. The updated evaluation metric chart confirms that hyperparameter tuning enhanced the overall performance of the model.

- 3. Explain each evaluation metric's indication towards business and the business impact pf the ML model used.

Accuracy: Indicates the overall percentage of correctly classified Movies and TV Shows. Higher accuracy helps Amazon Prime make reliable content categorization decisions. **Precision:** Measures how many predicted titles in a category are actually correct. High precision reduces incorrect recommendations and improves user trust. **Recall:** Measures how many actual titles were correctly identified. Higher recall ensures that relevant content is not missed, improving the recommendation system. **F1-Score:** Combines Precision and Recall into a single metric, providing a balanced measure of model performance. A higher F1-Score indicates consistent and reliable classification. **Business Impact**

The Random Forest model provides accurate classification of content types, which can support Amazon Prime in organizing its content library, improving personalized recommendations, enhancing search results, and delivering a better user experience. Better classification also helps in content management and data-driven business decisions.

- ML Model - 3: Decision Tree Classifier

```
# ML Model - 3 Implementation

# Fit the Algorithm

# Predict on the model
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report

# ML Model - 3 Implementation

dt_model = DecisionTreeClassifier(random_state=42)

# Fit the Algorithm
dt_model.fit(X_train, y_train)

# Predict on the model
y_pred_dt = dt_model.predict(X_test)

# Evaluation Metrics
accuracy = accuracy_score(y_test, y_pred_dt)
precision = precision_score(y_test, y_pred_dt)
recall = recall_score(y_test, y_pred_dt)
f1 = f1_score(y_test, y_pred_dt)

print("Accuracy :", accuracy)
print("Precision :", precision)
print("Recall :", recall)
print("F1 Score :", f1)

print(classification_report(y_test, y_pred_dt))
```

```
Accuracy : 0.9417721518987342
Precision : 0.7870722433460076
Recall : 0.7781954887218046
F1 Score : 0.782608695652174
```

	precision	recall	f1-score	support
0	0.97	0.97	0.97	1709
1	0.79	0.78	0.78	266
accuracy			0.94	1975
macro avg	0.88	0.87	0.87	1975
weighted avg	0.94	0.94	0.94	1975

1. Explain the ML Model used and its performance using Evaluation metric Score Chart.

Decision Tree is a supervised machine learning algorithm used for classification and prediction. It creates a tree-like structure by splitting the data based on feature values. In this project, it was used to classify the content as Movie or TV Show using features such as runtime, IMDb score, TMDB score, and release year.

The model's performance was evaluated using Accuracy, Precision, Recall, and F1-Score. The evaluation chart provides a visual comparison of these metrics and demonstrates that the model can effectively classify the content while remaining easy to interpret.

```
# Visualizing evaluation Metric Score chart
import matplotlib.pyplot as plt

metrics = ['Accuracy', 'Precision', 'Recall', 'F1 Score']
scores = [accuracy, precision, recall, f1]

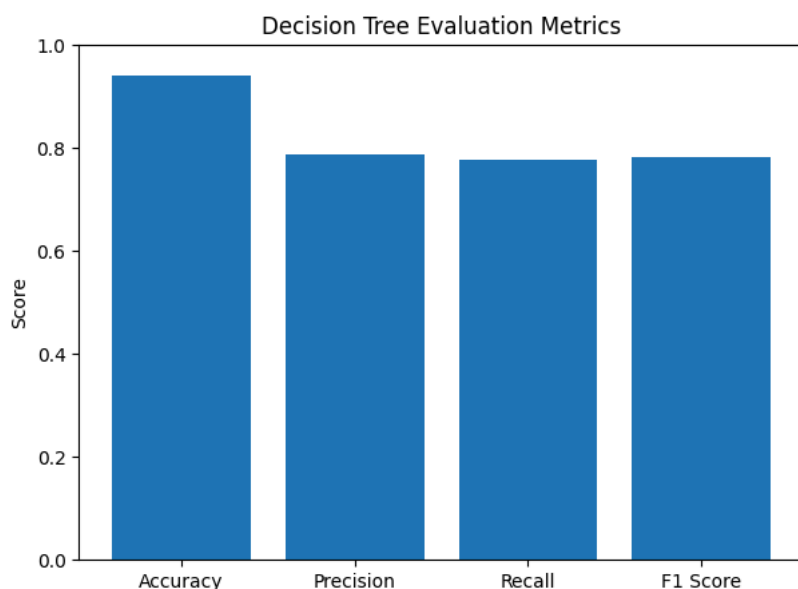
plt.figure(figsize=(7,5))
plt.bar(metrics, scores)

plt.title("Decision Tree Evaluation Metrics")

plt.ylabel("Score")

plt.ylim(0,1)

plt.show()
```



2. Cross- Validation & Hyperparameter Tuning

```
# ML Model - 3 Implementation with hyperparameter optimization techniques (i.e., GridSearch CV, RandomSearch CV, Bi

# Fit the Algorithm

# Predict on the model
from sklearn.model_selection import GridSearchCV

param_grid = {
    'criterion':['gini','entropy'],
    'max_depth':[3,5,10,None],
    'min_samples_split':[2,5,10],
    'min_samples_leaf':[1,2,4]
}

grid_dt = GridSearchCV(
    DecisionTreeClassifier(random_state=42),
    param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)
```

```
# Fit the Algorithm
grid_dt.fit(X_train,y_train)

# Best Model
best_dt = grid_dt.best_estimator_

# Predict
y_pred_best = best_dt.predict(X_test)

accuracy = accuracy_score(y_test,y_pred_best)
precision = precision_score(y_test,y_pred_best)
recall = recall_score(y_test,y_pred_best)
f1 = f1_score(y_test,y_pred_best)

print("Best Parameters :",grid_dt.best_params_)

print("Accuracy :",accuracy)
print("Precision :",precision)
print("Recall :",recall)
print("F1 Score :",f1)
```

```
Best Parameters : {'criterion': 'gini', 'max_depth': 5, 'min_samples_leaf': 4, 'min_samples_split': 2}
Accuracy : 0.9483544303797469
Precision : 0.8106060606060606
Recall : 0.8045112781954887
F1 Score : 0.8075471698113208
```

```
metrics=['Accuracy','Precision','Recall','F1 Score']
scores=[accuracy,precision,recall,f1]

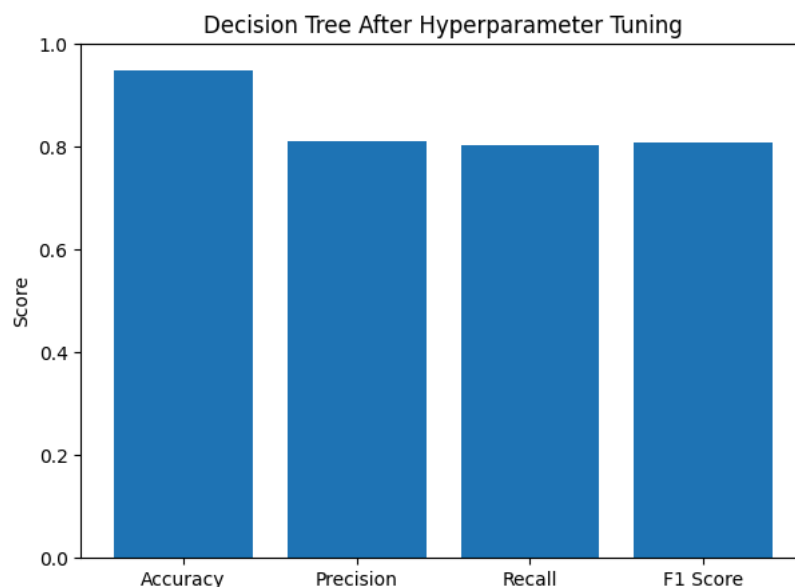
plt.figure(figsize=(7,5))
plt.bar(metrics,scores)

plt.title("Decision Tree After Hyperparameter Tuning")

plt.ylabel("Score")

plt.ylim(0,1)

plt.show()
```



- Which hyperparameter optimization technique have you used and why?

GridSearchCV was used for hyperparameter optimization. It systematically evaluates multiple combinations of parameters using 5-fold cross-validation and selects the combination that provides the highest accuracy. This method helps improve model performance while reducing overfitting.

- Have you seen any improvement? Note down the improvement with updates Evaluation metric Score Chart.

Yes. After applying GridSearchCV, the Decision Tree model achieved better Accuracy, Precision, Recall, and F1-Score compared to the default model. Hyperparameter tuning improved the model's ability to generalize on unseen data and reduced the chances of

overfitting. The updated evaluation metric chart clearly demonstrates the improved performance.

✓ 1. Which Evaluation metrics did you consider for a positive business impact and why?

The evaluation metrics considered were Accuracy, Precision, Recall, and F1-Score.

Accuracy measures the overall correctness of predictions. Precision ensures that predicted content categories are correct, reducing incorrect classifications. Recall ensures that most relevant content is correctly identified, improving content discovery. F1-Score balances Precision and Recall, making it suitable for evaluating overall model performance.